Parallel-batching scheduling with two agents, release dates and equal processing times

Shi-Sheng Li[a,*], Ren-Xia Chen[a]

[a]School of Mathematics and Information Science, Zhongyuan University of Technology, Zhengzhou 450007, People's Republic of China

Abstract

This paper investigates a two-agent scheduling problem with release dates and equal processing times on an unbounded parallel-batching machine. Each agent's scheduling criterion is regular and takes either the max-form or the sum-form. We address three variants of the problem. The first variant is the restricted version, which aims to determine a feasible schedule that minimizes one agent's criterion while keeping the other agent's objective value not exceeding a specified upper bound. The second variant is the linear-combination version, which seeks to find a feasible schedule that minimizes a linear combination of the criteria of both agents. The third variant is the Pareto version, which wants to identify all Pareto-optimal points and generate the corresponding Pareto-optimal schedules. When one agent's criterion is of the max-form, we present polynomial-time algorithms for all three variants. However, when both agents' criteria are of the sum-form, several variants of the problem are shown to be $\mathcal{NP}$-hard. In the Pareto version under this scenario, we design a pseudo-polynomial-time algorithm and a $(1, 1+\epsilon)$-approximate Pareto-optimal frontier. For the restricted version, we provide a fully polynomial-time approximation scheme (FPTAS) as well as a super-dual FPTAS.

Keywords: Two-agent scheduling; Parallel-batching; Release date; Equal processing times; Approximation scheme

1 Introduction

In smart manufacturing, modern production systems are increasingly complex, dynamic, and decentralized. Traditional scheduling approaches, which rely on centralized decision-making,

*Corresponding author. E-mail address: shishengli96@163.com (S.S. Li)

often struggle to address the challenges such as real-time disruptions, heterogeneous resource constraints, and competing objectives in contemporary industrial environments. Multi-agent scheduling (MAS) addresses these limitations through distributed intelligence, autonomous negotiation, and collaborative solutions. In a general MAS framework, multiple agents utilize shared processing resources to manage their respective sets of jobs, which may not be disjoint. Each agent desires to optimize its scheduling criterion by minimizing or maximizing specific objective functions, based on its assigned tasks. Applications of MAS span diverse domains, including packet-switching networks (Perez-Gonzalez and Framinan, 2014), cross-docking distribution (Agnetis et al., 2014; Kovalyov et al., 2015), production and logistics (Nouiri et al., 2020; Cai et al., 2023), cloud computing and manufacturing (Wang et al., 2021; Wang et al., 2024), and resource-constrained project scheduling (Servranckx et al., 2022; Geng and Yuan, 2023).

The use of parallel-batching (p-batch) processing has become prevalent in industrial manufacturing due to its ability to eliminate setup times/costs while streamlining production processes. Notable implementations include semiconductor manufacturing processes such as wafer fabrication (diffusion, deposition, oxidation) and burn-in operations in test facilities (Lee et al., 1992; Rania and Mathirajan, 2023; Kong et al., 2024), heat treatments in ceramic and steel industries (Li et al., 2022; Fan et al., 2023), food freezing processes (Kucukkoc et al., 2024), and fabric dyeing operations (Lei and Dai, 2024). In a typical p-batch production system, a parallel-batching processing machine (PBPM) can process multiple jobs simultaneously. Jobs within the same batch share identical starting and completion times, and the processing time of the batch is determined by the maximum processing time of jobs assigned to it.

Scheduling with equal processing times is a common strategy in repetitive manufacturing environments, particularly for large-scale production of standardized or similar items (Kravchenko and Werner, 2011; Oron et al., 2015). In real-world scenarios involving different job processing times, models that assume equal processing times can serve as critical benchmarks for evaluating algorithmic efficiency and identifying structural properties in scheduling systems. Moreover, algorithms designed for equal-duration tasks can generate lower bounds or approximate schedules for problems with different durations by decomposing tasks into a pseudo-polynomial number of equal-duration components (Dover and Shabtay, 2016).

In this paper, we explore a parallel-batching scheduling problem involving two competing agents, different job release dates, and equal processing times. As a practical application, we consider the automotive manufacturing sector (Streitberger and Dossel, 2008; Akafuah et al., 2016), where a coating station (machine) is utilized to apply protective layers to various car parts (jobs), including engine components, chassis parts, and body panels. To maximize throughput and efficiency, the coating station operates as a p-batch machine, capable of processing multiple parts simultaneously within a single batch. Parts arrive dynamically from upstream processes, such as machining or assembly, which results in arbitrary release

dates. Each part has a specified delivery date (due date) by which it is expected to be completed and delivered. The processing time for each part is uniform, as the coating process requires a fixed duration regardless of the part's size or type. These car parts are designated for two customers (agents). The first customer wants to minimize the maximum tardiness of their parts to ensure timely delivery of critical components and avoid delays in downstream assembly processes. In contrast, the second customer seeks to minimize the total tardiness of their parts to enhance production efficiency and reduce inventory holding costs. Consequently, the decision-maker should devise a scheduling strategy that effectively balances the objective functions of both customers: minimizing the maximum tardiness of the first customer while minimizing the total tardiness of the second customer as well. This context can be formulated as a parallel-batching scheduling problem involving two competing agents, dynamic job arrivals, and equal processing times. Similar applications also exist in other fields such as pharmaceutical production (Misra and Maravelias, 2022), semiconductor manufacturing (Lee and Uzsoy, 1999; Rania and Mathirajan, 2023), and fabric dyeing processes in the clothing industry (Lei and Dai, 2024).

1.1 Problem description

The scheduling problem under investigation is formally defined as follows: We have two disjoint groups of jobs, $\mathcal{J}^A = \{J_1^A, J_2^A, \ldots, J_{n_A}^A\}$ and $\mathcal{J}^B = \{J_1^B, J_2^B, \ldots, J_{n_B}^B\}$. For $X \in \{A, B\}$, the jobs in $\mathcal{J}^X$ are referred to as X-jobs and associated with agent X. Let $\mathcal{J} = \mathcal{J}^A \cup \mathcal{J}^B$ denote the set of all jobs, and let $n = n_A + n_B$ represent the number of jobs in $\mathcal{J}$. All n jobs have equal integer processing times, denoted by p. For $X \in \{A, B\}$ and $j \in \{1, 2, \ldots, n_X\}$, each job J_j^X has an integer release date $r_j^X \geq 0$, an integer due date $d_j^X \geq 0$, and an integer weight $w_j^X \geq 0$. For simplicity, we denote a job in $\mathcal{J}$ by J_j, with r_j, d_j, and w_j representing its release time, due date, and weight, respectively. The processing of jobs occurs on a single PBPM, which can handle up to b jobs simultaneously in a single batch. The processing time $p(\mathcal{B})$ of a batch $\mathcal{B}$ is determined by the maximum processing time of jobs assigned to it. There are two variants of the p-batch model regarding the batch size b: the bounded variant ($b < n$) and the unbounded variant ($b = n$). We focus on the scenario in which the batch size is unbounded. Moreover, we assume compatibility between the two agents, meaning that jobs from different agents can be processed together in the same batch.

A feasible schedule (or solution) for the n jobs in $\mathcal{J}$ is defined by a batch processing sequence $\mathcal{BPS} = (\mathcal{B}_1, \mathcal{B}_2, \ldots, \mathcal{B}_N)$, where each batch $\mathcal{B}_k$ is characterized by the set of jobs assigned to it and its starting time $S(\mathcal{B}_k)$. It is required that $S(\mathcal{B}_k) \geq r_{\max}(\mathcal{B}_k) = \max\{r_j, J_j \in \mathcal{B}_k\}$, where $r_{\max}(\mathcal{B}_k)$ denotes the release date of $\mathcal{B}_k$. The completion time of $\mathcal{B}_k$ is given by $C(\mathcal{B}_k) = S(\mathcal{B}_k) + p$. Given a feasible schedule ρ, let $S_j(\rho)$ ($S_j^X(\rho)$) denote the starting time and $C_j(\rho)$ ($C_j^X(\rho)$) denote the completion time of job J_j (J_j^X).

The evaluation of a schedule's quality involves two scheduling criteria, each pertaining to a different agent. For $X \in \{A, B\}$, let γ^X be the objective function of agent X to be minimized. Since the two agents are competing, γ^X is solely related to the completion times of the X-jobs. An objective function γ^X is referred to as *regular* if it is non-decreasing in the completion times C_j^X, $j = 1, 2, \ldots, n_X$. We examine two forms of regular objective functions: the *max-form* $f_{\max}^X = \max\{f_j^X(C_j^X) : j = 1, 2, \ldots, n_X\}$, and the *sum-form* $f_{\text{sum}}^X = \sum_{j=1}^{n_X} f_j^X(C_j^X)$, where $f_j^X(C_j^X)$ is non-decreasing in C_j^X. Special cases of $f_j^X(C_j^X)$ under consideration include C_j^X, L_j^X, T_j^X, U_j^X and Y_j^X. For each job J_j^X, the following definitions apply: $L_j^X = C_j^X - d_j^X$ indicates the lateness, $T_j^X = \max\{0, C_j^X - d_j^X\}$ represents the tardiness, $Y_j^X = \min\{\max\{0, C_j^X - d_j^X\}, p\}$ denotes the late work, and U_j^X is the tardy indicator, where $U_j^X = 1$ if $T_j^X > 0$ and $U_j^X = 0$ if $T_j^X = 0$. Furthermore, given a feasible schedule ρ, job J_j^X is classified as (i) *early* if $Y_j^X(\rho) = 0$; (ii) *tardy* if $T_j^X(\rho) > 0$; (iii) *late* if $Y_j^X(\rho) = p$; and (iv) *non-late* if $Y_j^X(\rho) < p$.

According to the scheduling framework and notation established in T'kindt and Billaut (2006) and Agnetis et al. (2014), we employ the following approach to characterize variants of two-agent scheduling problems. Here, α denotes the machine configuration, β includes additional problem-specific constraints, and Q_X is a specified upper bound on γ^X.

- Linear-combination version $\mathbf{V}_1$: $\alpha|\beta|\lambda\gamma^A + (1-\lambda)\gamma^B$. The aim is to determine a feasible schedule that minimizes $\lambda\gamma^A + (1-\lambda)\gamma^B$, where $0 \leq \lambda \leq 1$ is a constant.

- Decision version $\mathbf{V}_2$: $\alpha|\beta, \gamma^A \leq Q_A, \gamma^B \leq Q_B|-$. The aim is to verify the existence of a feasible schedule satisfying $\gamma^A \leq Q_A$ and $\gamma^B \leq Q_B$.

- Restricted version $\mathbf{V}_3$: $\alpha|\beta|\epsilon(\gamma^A, \gamma^B)$. The aim is to determine a feasible schedule that minimizes γ^A subject to $\gamma^B \leq Q_B$.

- Restricted version $\mathbf{V}_4$: $1|\beta|\epsilon(\gamma^B, \gamma^A)$. The aim is to determine a feasible schedule that minimizes γ^B subject to $\gamma^A \leq Q_A$.

- Pareto version $\mathbf{V}_5$: $\alpha|\beta|\#(\gamma^A, \gamma^B)$. The aim is to identify all Pareto-optimal points and generate the corresponding Pareto-optimal schedules. A feasible schedule ρ is referred to as *Pareto-optimal* with respect to γ^A and γ^B if there exists no other schedule ρ' with $\gamma^A(\rho') \leq \gamma^A(\rho)$ and $\gamma^B(\rho') \leq \gamma^B(\rho)$, where at least one of the two inequalities is strict. In this context, $(\gamma^A(\rho), \gamma^B(\rho))$ is referred to as a *Pareto-optimal point*. A set of $\alpha|\beta|\#(\gamma^A, \gamma^B)$ is referred to as a *Pareto-optimal frontier* (POF) if it contains all Pareto-optimal points.

To simplify the discussion, we use the notation $\alpha|\beta|\{\gamma^A, \gamma^B\}$ to denote the five variants of the general problem. Following the observations in T'kindt and Billaut (2006), we note that: (i) the algorithm designed for $\mathbf{V}_5$ can solve $\mathbf{V}_1$-$\mathbf{V}_4$ as a by-product; (ii) $\mathbf{V}_2$ is the decision version of both $\mathbf{V}_3$ and $\mathbf{V}_4$, which implies that either neither $\mathbf{V}_3$ nor $\mathbf{V}_4$ is $\mathcal{NP}$-hard, or both

are $\mathcal{NP}$-hard; (iii) $\mathbf{V}_3$ and $\mathbf{V}_4$ are computationally at least as difficult as $\mathbf{V}_2$; (iv) if any one variant among $\mathbf{V}_2$-$\mathbf{V}_4$ admits a pseudo-polynomial-time algorithm, then the other two can also be solved in pseudo-polynomial time; (v) for any $\mathbf{V}_i$ $(i = 1, 2, 3, 4)$, the existence of a feasible schedule guarantees the existence of a Pareto-optimal schedule.

For convenience, we denote "p-batch, $b = n, r_j, p_j = p$" in the β field as β^* in subsequent discussions. Accordingly, the decision version of the problem under study is denoted by $1|\beta^*, \gamma^A \leq Q_A, \gamma^B \leq Q_B|-$, while the other variants are denoted by $1|\beta^*|\gamma$, where γ is substituted by $\{\gamma^A, \gamma^B\}$ for the general problem, and by $\lambda\gamma^A+(1-\lambda)\gamma^B$, $\epsilon(\gamma^A, \gamma^B)$, $\epsilon(\gamma^B, \gamma^A)$, and $\#(\gamma^A, \gamma^B)$ for the variants $\mathbf{V}_1$, $\mathbf{V}_3$, $\mathbf{V}_4$, and $\mathbf{V}_5$, respectively.

1.2 Literature review

The scheduling problem addressed in this research falls within the categories of parallel-batching scheduling and multi-agent scheduling. Given the extensive nature of problems related to these two categories, we review only some relevant works in the single-machine context.

The initial research on parallel-batching scheduling traces back to Ikura and Gimple (1986), who proposed an $O(n \log n)$-time algorithm for $1|p\text{-batch}, b < n, r_j, p_j = p|C_{\max}$. Lee et al. (1992) designed polynomial-time dynamic programming (DP) algorithms for $1|p\text{-batch}, b < n, p_j = p, arg(r_j, d_j)|T_{\max}$ and $1|p\text{-batch}, b < n, p_j = p, arg(r_j, d_j)|\sum U_j$, where $T_{\max} = \max\{T_j : 1 \leq j \leq n\}$ denotes the maximum tardiness, and $arg(r_j, d_j)$ indicates that the release dates and due dates of all jobs are agreeable (i.e., $r_i \leq r_j$ implies $d_i \leq d_j$ for each $i, j = 1, 2, \ldots, n$). Li and Lee (1997) demonstrated that problems $1|p\text{-batch}, b < n, arg(r_j, d_j)|T_{\max}$ and $1|p\text{-batch}, b < n, arg(r_j, d_j)|\sum U_j$ are both strongly $\mathcal{NP}$-hard. Additionally, they presented polynomial-time algorithms for $1|p\text{-batch}, b < n, arg(r_j, d_j, p_j)|T_{\max}$ and $1|p\text{-batch}, b < n, arg(r_j, d_j, p_j)|\sum U_j$, where $arg(r_j, d_j, p_j)$ denotes that the release dates, due dates, and processing times of all jobs are agreeable. Brucker et al. (1998) provided a systematic complexity classification for both bounded and unbounded versions of the single PBPM scheduling problem under various regular criteria. Lee and Uzsoy (1999) concentrated on minimizing the makespan for a single PBPM with dynamic job arrivals. They developed an $O(n^2)$-time algorithm for $1|p\text{-batch}, b = n, r_j|C_{\max}$ and a pseudo-polynomial-time algorithm for $1|p\text{-batch}, b < n, r_j|C_{\max}$ with two release dates. They also proposed several heuristics and evaluated their effectiveness through numerical testing. Baptiste (2000) provided polynomial-time algorithms for $1|p\text{-batch}, b < n, p_j = p, r_j|\gamma$, where $\gamma \in \{\sum w_j U_j, \sum w_j C_j, \sum T_j\}$. Cheng et al. (2001) demonstrated that problem $1|p\text{-batch}, b = n, r_j, arg(p_j, d_j)|L_{\max}$ is binary $\mathcal{NP}$-hard and designed polynomial-time algorithms for several special cases. Liu et al. (2003) showed that problem $1|p\text{-batch}, b = n|\sum T_j$ is $\mathcal{NP}$-hard and proposed pseudo-polynomial-time algorithms for $1|p\text{-batch}, b = n, r_j|f_{\max}$ and $1|p\text{-batch}, b = n, r_j|f_{\text{sum}}$. Cheng et al. (2005)

investigated an unbounded PBPM scheduling involving precedence constraints, dynamic job arrivals, and equal processing times, aiming to minimize a regular criterion γ. They designed an $O(n^3)$-time algorithm for $1|p\text{-batch}, b = n, r_j, p_j = p|\gamma$ and developed a $\frac{3}{2}$-approximation algorithm for $1|prec, p\text{-batch}, b = n, r_j, p_j = p|\sum w_j C_j$. Liu et al. (2007) established that problem $1|p\text{-batch}, b = 2, arg(p_j, d_j)|\sum T_j$ is $\mathcal{NP}$-hard and problems $1|p\text{-batch}, b < n, arg(p_j, d_j)|\sum T_j$ and $1|p\text{-batch}, b < n, arg(p_j, d_j)|\sum w_j U_j$ are pseudo-polynomially solvable. Recent studies on parallel-batching scheduling problems have explored various themes, such as preventive maintenance (Huang et al., 2020), non-identical job sizes (Zhou et al., 2022), energy-aware objective (Kucukkoc et al., 2024), and energy conservation concerns (Kong et al., 2024). For a more comprehensive overview of this area, the readers are directed to the reviews by Monch et al. (2011) and Fowler and Monch (2022).

The set of MAS models stems from two seminal papers by Baker and Smith (2003) and Agnetis et al. (2004), which pioneered the analysis of two-agent scheduling scenarios. For various combinations of scheduling criteria between the two agents, Baker and Smith (2003) investigated the linear-combination version, while Agnetis et al. (2004) analyzed the restricted and Pareto variants. Agnetis et al. (2014) and Perez-Gonzalez and Framinan (2014) established a comprehensive framework and taxonomy for MAS literature, systematically analyzing computational complexity and solution methods in studies published through 2014. MAS model demonstrates broad applicability across diverse domains, as evidenced by the variety of models explored in the literature. Recent studies on two-agent scheduling models have investigated topics, such as equal processing times (Zhao and Yuan, 2020; Chen et al., 2022), multitasking (Li et al., 2020; Wang et al., 2021), optional job rejection (Oron, 2021; Freud and Mosheiov, 2021), dynamic job arrivals (Li and Chen, 2023; Wang et al., 2024), rate-modifying activities (Phosavanh and Oron, 2024), and the price of fairness (Agnetis et al., 2019; Yu et al., 2025).

There are also some studies focusing on two-agent scheduling in the context of parallel-batching. Sabouni and Jolai (2010) examined a single PBPM scheduling problem in which one agent's objective function is the makespan and the other agent's objective function is the maximum lateness. They designed polynomial-time algorithms for $1|p\text{-batch}, b = n, IG|\lambda C^A_{\max} + (1-\lambda)L^B_{\max}$, $1|p\text{-batch}, b < n, p^B_j = p^B, IG|\lambda C^A_{\max} + (1-\lambda)L^B_{\max}$, and $1|p\text{-batch}, b < n, p^B_j = p^B, CG|\#(C^A_{\max}, L^B_{\max})$, where $p^B_j = p^B$ means that all B-jobs have equal processing times, IG denotes incompatibility between the two agents (i.e., jobs from different agents cannot be handled together in the same batch), and CG indicates compatibility (i.e., jobs from different agents can be handled together in the same batch). Li and Yuan (2012) addressed the restricted variant of two-agent scheduling problem on a single unbounded PBPM. They demonstrated that: (i) problems $1|p\text{-batch}, b = n, \phi|\epsilon(f^A_{\max}, f^B_{\max})$ and $1|p\text{-batch}, b = n, \phi|\epsilon(\sum U^A_j, \sum U^B_j)$ are polynomially solvable, (ii) problems $1|p\text{-batch}, b = n, \phi|\epsilon(\sum w^A_j C^A_j, C^B_{\max})$ and $1|p\text{-batch}, b = n, \phi|\epsilon(\sum w^A_j C^A_j, \sum C^B_j)$ are binary $\mathcal{NP}$-hard, and (iii) problems $1|p\text{-batch}, b = n, \phi|\epsilon(\sum f^A_j, f^B_{\max})$ and $1|p\text{-batch}, b = n, \phi|\epsilon(\sum f^A_j, \sum f^B_j)$ are

pseudo-polynomially solvable, where $\phi \in \{IG, CG\}$. Fan et al. (2013) studied the bounded PBPM variant of two-agent scheduling, establishing the following results: (i) problem $1|p\text{-batch}, b < n, IG|\epsilon(C^A_{\max}, C^B_{\max})$ is solvable in $O(n \log n)$ time, (ii) problem $1|p\text{-batch}, b < n, CG|\epsilon(C^A_{\max}, C^B_{\max})$ is $\mathcal{NP}$-hard and admits a pseudo-polynomial-time solution, and (iii) problem $1|p\text{-batch}, b < n, CG|\epsilon(\sum C^A_j, C^B_{\max})$ is $\mathcal{NP}$-hard. Tang et al. (2017) investigated the restricted variant of two-agent scheduling problem on a single PBPM, analyzing both bounded and unbounded batch sizes with time-dependent processing times that deteriorate in a proportional-linear manner. Depending on the objective function combinations of the two agents, the problem was shown to be either $\mathcal{NP}$-hard or solvable by an exact algorithm. Focusing on the goal of minimizing the makespan for both agents, Gao et al. (2019) revisited the work of Tang et al. (2017) and derived new theoretical insights. He et al. (2022) introduced a polynomial-time algorithm for $1|p\text{-batch}, b < n, p^B_j = p^B, CG|\#(C^A_{\max}, f^B_{\max})$.

To the best of our knowledge, none of the studies mentioned above or their references have investigated the issue of two-agent scheduling in a parallel-batching environment with dynamic job arrivals. This gap is particularly significant given the increasing complexity of scheduling problems in modern manufacturing systems. Considering this issue, our research represents the first attempt to integrate parallel-batching scheduling, two competing agents, job release dates, and equal processing times. The primary focus of this research is twofold: (i) to ascertain the complexity status of various problem variants, and (ii) to develop efficient exact and approximation algorithms (schemes) for these $\mathcal{NP}$-hard problems.

The remainder of this paper is structured as follows. Section 2 presents some preliminary results for the general problem $1|\beta^*|\{\gamma^A, \gamma^B\}$. In Section 3, we analyze the problem when one agent's scheduling criterion is of the max-form, demonstrating that all problem variants can be solved in polynomial time. Section 4 investigates problem $1|\beta^*, f^A_{\text{sum}} \leq Q_A, f^B_{\text{sum}} \leq Q_B|-$ and establishes $\mathcal{NP}$-completeness results for various combinations of f^A_{sum} and f^B_{sum}. Section 5 introduces an exact DP algorithm for $1|\beta^*|\#(f^A_{\text{sum}}, f^B_{\text{sum}})$. In Section 6, we address problem $1|\beta^*|\epsilon(f^A_{\text{sum}}, f^B_{\text{sum}})$ and propose three algorithmic approaches: (i) an exact pseudo-polynomial-time algorithm with time complexity $O(n^4 Q_B)$, (ii) an FPTAS with time complexity $O(n^5(\log\log n_A + 1/\epsilon))$, and (iii) a super-dual FPTAS with time complexity $O(n^5/\epsilon)$. Section 7 devises a $(1, 1+\epsilon)$-approximate POF for $1|\beta^*|\#(f^A_{\text{sum}}, f^B_{\text{sum}})$. Section 8 evaluates the proposed algorithms through numerical experiments. In Section 9, we provide some managerial insights. Section 10 concludes the paper and discusses future research directions.

2 Preliminaries

We start with providing some important properties that can be applied to the general problem $1|\beta^*|\{\gamma^A, \gamma^B\}$ and their various special scenarios.

Since the objective functions γ^A and γ^B under study are both regular, we only concentrate

on the schedules in which each batch starts as early as possible. Hence, we obtain the following lemma.

Lemma 2.1 *If $\mathcal{BPS} = (\mathcal{B}_1, \mathcal{B}_2, \ldots, \mathcal{B}_N)$ is a feasible schedule for $1|\beta^*|\{\gamma^A, \gamma^B\}$, then we have $S(\mathcal{B}_1) = r(\mathcal{B}_1)$, $S(\mathcal{B}_i) = \max\{S(\mathcal{B}_{i-1}) + p, r(\mathcal{B}_i)\}$ for $i = 2, 3, \ldots, N$.*

In the remainder of this paper, the n jobs in $\mathcal{J}$ are renumbered in line with the earliest release date (ERD) rule, i.e.

$$r_1 \leq r_2 \leq \cdots \leq r_n. \tag{1}$$

A schedule is called an *ERD-batch schedule* if, for any $J_h \in \mathcal{B}_i$ and $J_k \in \mathcal{B}_{i+1}$, $r_h < r_k$ holds. The following result demonstrates that a feasible schedule for $1|\beta^*|\{\gamma^A, \gamma^B\}$ can be selected among all Pareto-optimal ERD-batch schedules.

Lemma 2.2 *If a feasible schedule exists for $1|\beta^*|\{\gamma^A, \gamma^B\}$, then a Pareto-optimal ERD-batch schedule also exists.*

Proof. Let σ be a Pareto-optimal schedule for $1|\beta^*|\{\gamma^A, \gamma^B\}$, but not an ERD-batch schedule. Then for some i, there exist two jobs $J_h \in \mathcal{B}_i$ and $J_k \in \mathcal{B}_{i+1}$ in σ such that $r_h \geq r_k$. We construct another schedule σ' from σ by moving J_k to $\mathcal{B}_i$. Based on Lemma 2.1 and the fact that $r_k \leq r_h \leq r(\mathcal{B}_i)$, we have $C_k(\sigma') < C_k(\sigma)$ and $C_j(\sigma') \leq C_j(\sigma)$ for each $J_j \in \mathcal{J} \setminus \{J_k\}$. Since γ^A and γ^B are both regular, we have $\gamma^A(\sigma') \leq \gamma^A(\sigma)$ and $\gamma^B(\sigma') \leq \gamma^B(\sigma)$. Therefore, σ' is a Pareto-optimal schedule as well. By iteratively repeating this procedure, we can construct a feasible schedule with the desired form. □

By Lemma 2.2, an optimal schedule exists where jobs from both agents with identical release dates are scheduled in the same batch. Furthermore, due to the unbounded batch size, the starting time of the last batch in such a schedule does not exceed $r_{\max}(\mathcal{J}) + p - 1$. Write $\Lambda = r_{\max}(\mathcal{J}) + 2p - 1$, and define

$$\mathcal{R} = \{r_j + k_j p : r_j + k_j p \leq \Lambda, j = 1, 2, \ldots, n; k_j = 1, 2, \ldots, n - j + 1\}. \tag{2}$$

Let $R_1 < R_2 < \cdots < R_m$ denote the ordered sequence of distinct members in $\mathcal{R}$. Since $m = |\mathcal{R}| = O(n^2)$, this sequence can be obtained in $O(n^2 \log n)$ time. Moreover, define

$$\mathcal{V}_X = \{f_j^X(R_k) : j = 1, 2, \ldots, n_X, k = 1, 2, \ldots, m\},\ X \in \{A, B\}. \tag{3}$$

Based on Lemmas 2.1 and 2.2, for $1|\beta^*|\{\gamma^A, \gamma^B\}$, we focus on optimal schedules in which each batch' completion time belongs to the set $\mathcal{R}$. Consequently, $\mathcal{V}_X$ denotes the set of all possible $f_{\max}^X$ values of agent X. From the fact that $|\mathcal{R}| = O(n^2)$, the following lemma holds.

Lemma 2.3 *For $X \in \{A, B\}$, the number of different possible $f_{\max}^X$ values of agent X is bounded by $O(n^2 n_X)$.*

When all jobs belong to a single agent, Cheng et al. (2005) establishes the following result, which plays a critical role in our subsequent discussion.

Lemma 2.4 *(Cheng et al., 2005) Problem $1|\beta^*|\gamma$ is solvable in $O(n^3)$ time, where γ is an arbitrary regular scheduling criterion.*

The DP algorithm proposed by Cheng et al. (2005) has a space complexity of $O(n^3)$, as it must store all necessary states and variables during execution.

Theorem 2.1 *Problem $1|\beta^*|\lambda f^A_{\rm sum} + (1-\lambda) f^B_{\rm sum}$ is solvable in $O(n^3)$ time.*

Proof. Note that problem $1|\beta^*|\lambda f^A_{\rm sum} + (1-\lambda) f^B_{\rm sum}$ can be reduced in linear time to problem $1|\beta^*|f_{\rm sum}$ over the same job set $\mathcal{J}$. Here, $f_{\rm sum} = \sum_{j=1}^{n} f_j(C_j)$ is defined such that for each $J_j \in \mathcal{J}$, $f_j(t) = \lambda f^A_j(t)$ if $J_j \in \mathcal{J}^A$, and $f_j(t) = (1-\lambda) f^B_j(t)$ if $J_j \in \mathcal{J}^B$. By Lemma 2.4, the theorem follows directly. □

Let $\widehat{r}_1 < \widehat{r}_2 < \cdots < \widehat{r}_h$ denote the ordered sequence of distinct release dates of jobs in $\mathcal{J}$. When $p_j = 1$ for each $J_j \in \mathcal{J}$, Lemma 2.1 implies that $\mathcal{BPS} = (\mathcal{B}_1, \mathcal{B}_2, \ldots, \mathcal{B}_h)$ is an optimal schedule for $1|\beta^*, p=1|\{\gamma^A, \gamma^B\}$, where $\mathcal{B}_k = \{J_j \in \mathcal{J} : r_j = \widehat{r}_k\}$ for each $k = 1, 2, \ldots, h$. This leads to the following theorem.

Theorem 2.2 *Problem $1|\beta^*, p=1|\{\gamma^A, \gamma^B\}$ is solvable in $O(n \log n)$ time.*

3 Problems with $\gamma^X = f^X_{\max}$ for some agent $X \in \{A, B\}$

In this section, we address the problem when one agent's scheduling criterion is of the max-form, demonstrating that all problem variants can be solved in polynomial time.

3.1 Problem $1|\beta^*|\epsilon(\gamma^A, f^B_{\max})$

By borrowing the approach used in Agnetis et al. (2004) for $1||\epsilon(f^A_{\max}, f^B_{\max})$, we show that $1|\beta^*|\epsilon(\gamma^A, f^B_{\max})$ can be solved through a reduction to $1|\beta^*|\gamma$, which was studied by Cheng et al. (2005).

For $\gamma^A = f^A_{\max}$, define $\gamma = \max\{f_j(C_j) : j = 1, 2, \ldots, n\}$, where, for each $t \geq 0$,

$$f_j(t) = \begin{cases} f^A_j(t), & \text{if } J_j \in \mathcal{J}^A; \\ -\infty, & \text{if } J_j \in \mathcal{J}^B \text{ and } f^B_j(t) \leq Q_B; \\ +\infty, & \text{if } J_j \in \mathcal{J}^B \text{ and } f^B_j(t) > Q_B. \end{cases}$$

For $\gamma^A = f^A_{\max}$, define $\gamma = \sum_{j=1}^{n} f_j(C_j)$, where, for each $t \geq 0$,

$$f_j(t) = \begin{cases} f_j^A(t), & \text{if } J_j \in \mathcal{J}^A; \\ 0, & \text{if } J_j \in \mathcal{J}^B \text{ and } f_j^B(t) \leq Q_B; \\ +\infty, & \text{if } J_j \in \mathcal{J}^B \text{ and } f_j^B(t) > Q_B. \end{cases}$$

If ρ^* is an optimal schedule for $1|\beta^*|\gamma$ with $\gamma(\rho^*) < +\infty$, then ρ^* is also an optimal schedule for $1|\beta^*|\epsilon(\gamma^A, f^B_{\max})$ and $\gamma^A(\rho^*) = \gamma(\rho^*)$. Otherwise, if $\gamma(\rho^*) = +\infty$, then $1|\beta^*|\epsilon(\gamma^A, f^B_{\max})$ has no feasible solution. Combining the above analysis with Lemma 2.4, we establish the following theorem.

Theorem 3.1 *Problems $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$ and $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$ are both solvable in $O(n^3)$ time.*

3.2 Problem $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\text{sum}})$

The following result is given by Oron et al. (2015), which characterizes an important symmetric property for $\alpha|\beta|\epsilon(\gamma^A, \gamma^B)$ and $\alpha|\beta|\epsilon(\gamma^B, \gamma^A)$.

Lemma 3.1 *(Oron et al., 2015) If there exists an optimal algorithm for $\alpha|\beta|\epsilon(\gamma^A, \gamma^B)$ with time complexity $O(T_2)$, then problem $\alpha|\beta|\epsilon(\gamma^B, \gamma^A)$ can be solved in $O(\max\{T_2, N_2\} \log N_2)$ time, where the number of different γ^B values does not exceed N_2.*

Based on the facts that: (i) problem $1|\beta^*|\epsilon(f^B_{\text{sum}}, f^A_{\max})$ is solvable in $O(n^3)$ time (Theorem 3.1), (ii) the number of different $f^A_{\max}$ values is bounded by $O(n^2 n_A)$ (Lemma 2.3), and (iii) the symmetric property (Lemma 3.1), we obtain the following theorem.

Theorem 3.2 *Problem $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\text{sum}})$ is solvable in $O(n^3 \log n)$ time.*

3.3 Problem $1|\beta^*, f^A_{\max} \leq Q_A, f^B_{\max} \leq Q_B|-$

The approach in Section 3.1 can be directly applied to solve $1|\beta^*, f^A_{\max} \leq Q_A, f^B_{\max} \leq Q_B|-$ and $1|\beta^*, f^A_{\text{sum}} \leq Q_A, f^B_{\max} \leq Q_B|-$ in $O(n^3)$ time. For the former problem, we next provide a faster $O(n^2)$-time algorithm by adapting the DP framework designed for $1|p\text{-batch}, b = n, r_j, arg(r_j, p_j), C_j \leq \overline{d}_j|-$ (Cheng et al., 2001), where $arg(r_j, p_j)$ denotes that the release dates and processing times of all jobs are agreeable.

Any ERD-batch schedule can be generated by first sorting jobs in ascending order of their indices and then partitioning the sorted sequence into batches. Let $M(j)$ denote the minimum makespan among all feasible ERD-batch schedules that consist of jobs $J_1, J_2, \ldots, J_j$. If no such schedules exist, define $M(j) = +\infty$. Following Lemma 2.2, the proposed approach schedules jobs $J_1, J_2, \ldots, J_{k-1}$ optimally for some k $(1 \leq k \leq j)$, then processes jobs

$J_k, J_{k+1}, \ldots, J_j$ as a single batch starting at time $\max\{M(k-1), r_j\}$. For each $j = 1, 2, \ldots, n$ and $t \geq 0$, define

$$m_j(t) = \begin{cases} 0, & \text{if } J_j \in \mathcal{J}^A \text{ and } f_j^A(t) \leq Q_A; \\ 0, & \text{if } J_j \in \mathcal{J}^B \text{ and } f_j^B(t) \leq Q_B; \\ +\infty, & \text{if } J_j \in \mathcal{J}^A \text{ and } f_j^A(t) > Q_A; \\ +\infty, & \text{if } J_j \in \mathcal{J}^B \text{ and } f_j^B(t) > Q_B. \end{cases}$$

The initial condition is $M(0) = 0$.

For $j = 1, 2, \ldots, n$, the recursive formula for $M(j)$ is defined as follows:

$$M(j) = \min\{M(j,k) : k = 1, 2, \ldots, j\},$$

where

$$M(j,k) = \begin{cases} \max\{M(k-1), r_j\} + p, & \text{if } \sum_{i=k}^{j} m_i(\max\{M(k-1), r_j\} + p) = 0; \\ +\infty, & \text{otherwise.} \end{cases}$$

The recursive procedure terminates if some $M(j) = +\infty$ is reached, indicating that no feasible schedule exists for $1|\beta^*, f_{\max}^A \leq Q_A, f_{\max}^B \leq Q_B|-$. If $M(n) < +\infty$, then the problem admits a feasible solution, and the associated schedule can be retrieved through backtracking. The time complexity of the above algorithm is $O(n^2)$, as each $M(j)$ computation requires $O(n)$ time and there are n iterations. Since all possible states (j,k) must be stored, the space complexity is also $O(n^2)$. This leads to the following theorem.

Theorem 3.3 *Problem $1|\beta^*, f_{\max}^A \leq Q_A, f_{\max}^B \leq Q_B|-$ is solvable in $O(n^2)$ time.*

3.4 Problem $1|\beta^*|\#(\gamma^A, f_{\max}^B)$

According to (3), we know that $\mathcal{V}_B = \{f_j^B(R_k) : j = 1, 2, \ldots, n_B, k = 1, 2, \ldots, m\}$. To solve $1|\beta^*|\#(\gamma^A, f_{\max}^B)$, we generate $|\mathcal{V}_B| = O(n^2 n_B)$ optimal schedules for instances of $1|\beta^*|\epsilon(\gamma^A, f_{\max}^B)$, where each instance corresponds to a distinct upper bound $Q_B \in \mathcal{V}_B$ for $f_{\max}^B$. Among these schedules, Pareto-optimal solutions are then extracted. Utilizing the results of Lemma 2.3 and Theorem 3.1, the following result holds.

Theorem 3.4 *Problems $1|\beta^*|\#(f_{\max}^A, f_{\max}^B)$ and $1|\beta^*|\#(f_{\text{sum}}^A, f_{\max}^B)$ are both solvable in $O(n^5 n_B)$ time.*

Since the algorithm developed for $1|\beta^*|\#(\gamma^A, f_{\max}^B)$ can be directly applied to $1|\beta^*|\lambda\gamma^A + (1-\lambda)f_{\max}^B$, the following two results hold.

Corollary 3.1 *Problem $1|\beta^*|\lambda f_{\max}^A + (1-\lambda)f_{\max}^B$ is solvable in $O(n^5 n_B)$ time.*

Corollary 3.2 *Problem $1|\beta^*|\lambda f_{\text{sum}}^A + (1-\lambda)f_{\max}^B$ is solvable in $O(n^5 n_B)$ time.*

4 $\mathcal{NP}$-completeness results

In this section, we demonstrate that some variants of the problem $1|\beta^*, f^A_{\max} \le Q_A, f^B_{\max} \le Q_B|-$ are $\mathcal{NP}$-completeness, where $f^X_{\max} \in \{\sum C^X_j, \sum w^X_j C^X_j, \sum Y^X_j, \sum T^X_j, \sum w^X_j U^X_j\}$ for $X \in \{A, B\}$. Notably, these $\mathcal{NP}$-completeness results remain valid for the case of bounded batch size (i.e., $b < n$), as shown in the proofs of the following theorems. All reductions are derived from the $\mathcal{NP}$-complete ***Partition*** problem (Garey and Johnson, 1979), which is described as follows:

Partition problem (PP): Given g positive integers $z_1, z_2, \ldots, z_g$ and an integer Z with $\sum_{j=1}^{g} z_j = 2Z$, determine whether the index set $\mathcal{I} = \{1, 2, \ldots, g\}$ can be partitioned into two disjoint subsets $\mathcal{I}_1$ and $\mathcal{I}_2$ with $\sum_{j\in\mathcal{I}_1} z_j = \sum_{j\in\mathcal{I}_2} z_j = Z$?

The following lemma establishes that the two objective functions $\sum Y^X_j$ and $\sum T^X_j$ play the same role under a certain condition.

Lemma 4.1 *Let ρ be a feasible schedule for $1|\beta^*, \sum f^A_j \le Q_A, \sum f^B_j \le Q_B|-$, and let $\sum f^X_j \in \{\sum Y^X_j, \sum T^X_j\}$ for some X ($X \in \{A, B\}$). If $\sum f^X_j(\rho) < p$, then each X-job is non-late in ρ, and so $f^X_k = Y^X_k = T^X_k$ for each $J^X_k \in \mathcal{J}^X$.*

Proof. If J^X_k is a late X-job in ρ, its completion time satisfies $C^X_k(\rho) \ge d^X_k + p$. This implies $T^X_k(\rho) \ge p$ and $Y^X_k(\rho) = p$. Consequently, $\sum f^X_j(\rho) \ge f^X_k(\rho) \ge p$, which contradicts the condition $\sum f^X_j(\rho) < p$. □

4.1 $\mathcal{NP}$-completeness of $1|\beta^*, \sum w^A_j C^A_j \le Q_A, \sum w^B_j C^B_j \le Q_B|-$

Given an instance $\Phi = (z_1, z_2, \ldots, z_g; Z)$ of **PP**, we build an instance Ψ of $1|\beta^*, \sum w^A_j C^A_j \le Q_A, \sum w^B_j C^B_j \le Q_B|-$ as follows:

- Each group $\mathcal{J}_X$ consists of g jobs $J^X_1, J^X_2, \ldots, J^X_g$, all with equal processing times $p = 2Z$, where $X = A, B$.
- Each job J^A_j, $j = 1, 2, \ldots, g$, has a release date $r^A_j = (4j-4)Z$ and a weight $w^A_j = z_j$.
- Each job J^B_j, $j = 1, 2, \ldots, g$, has a release date $r^B_j = (4j-3)Z$ and a weight $w^B_j = z_j$.
- The upper bounds of the two agents are $Q_A = \sum_{j=1}^{g} z_j(4j-2)Z + Z^2$ and $Q_B = \sum_{j=1}^{g} z_j(4j-1)Z + Z^2$, respectively.
- The decision is to find whether there is a schedule ρ (called *normal*) with $\sum_{j=1}^{g} w^A_j C^A_j(\rho) \le Q_A$ and $\sum_{j=1}^{g} w^B_j C^B_j(\rho) \le Q_B$.

Next, we demonstrate that Φ has a solution if and only if Ψ has a normal schedule.

Lemma 4.2 *If Φ has a solution, then Ψ has a normal schedule.*

Proof. Let $(\mathcal{I}_1, \mathcal{I}_2)$ be a solution to Φ. For each $j \in \mathcal{I}_1$, define $\mathcal{B}_j^1 = \{J_j^A\}$ and $\mathcal{B}_j^2 = \{J_j^B\}$. For each $j \in \mathcal{I}_2$, define $\mathcal{B}_j^1 = \{J_j^A, J_j^B\}$ and $\mathcal{B}_j^2 = \emptyset$. A schedule ρ for Ψ is defined by the batch processing sequence $\mathcal{BPS} = (\mathcal{B}_1^1, \mathcal{B}_1^2, \mathcal{B}_2^1, \mathcal{B}_2^2, \ldots, \mathcal{B}_g^1, \mathcal{B}_g^2)$, where $p(\emptyset) = 0$.

By Lemma 2.1, we obtain that (i) $C_j^A(\rho) = r_j^A + p = (4j-2)Z$ and $C_j^B(\rho) = C_j^A(\rho) + p = 4jZ$ for $j \in \mathcal{I}_1$, and (ii) $C_j^A(\rho) = C_j^B(\rho) = r_j^B + p = (4j-1)Z$ for $j \in \mathcal{I}_2$.

Since $\sum_{j\in\mathcal{I}_1} z_j = \sum_{j\in\mathcal{I}_2} z_j = Z$, it follows that (i) $\sum_{j=1}^g w_j^A C_j^A(\rho) = \sum_{j\in\mathcal{I}_1} z_j(4j-2)Z + \sum_{j\in\mathcal{I}_2} z_j(4j-1)Z = \sum_{j\in\mathcal{I}} z_j(4j-2)Z + \sum_{j\in\mathcal{I}_2} z_j Z = \sum_{j=1}^g z_j(4j-2)Z + Z^2 = Q_A$, and (ii) $\sum_{j=1}^g w_j^B C_j^B(\rho) = \sum_{j\in\mathcal{I}_1} z_j(4jZ) + \sum_{j\in\mathcal{I}_2} z_j(4j-1)Z = \sum_{j\in\mathcal{I}} z_j(4j-1)Z + \sum_{j\in\mathcal{I}_1} z_j Z = \sum_{j=1}^g z_j(4j-1)Z + Z^2 = Q_B$. Therefore, ρ is a normal schedule for Ψ. □

Lemma 4.3 *If Ψ has a normal schedule, then Φ has a solution.*

Proof. Let ρ be a normal ERD-batch schedule for Ψ (Lemma 2.2). From the definition of job parameters, we have

$$r_j^A < r_j^B < r_j^A + p < r_j^B + p < r_j^A + 2p = r_{j+1}^A,\ j \in \{1, 2, \ldots, g\},$$

where $r_{g+1}^A = 4gZ$. In view of Lemmas 2.1 and 2.2, the starting time of J_j^A in ρ is constrained to be either r_j^A or r_j^B. In the former case, J_j^B starts at time $r_j^A + p$. In the latter case, J_j^B starts at time r_j^B as well. Define $\mathcal{I}_1 = \{j : S_j^A(\rho) = r_j^A\}$ and $\mathcal{I}_2 = \{j : S_j^A(\rho) = r_j^B\}$. By simple computation, we obtain that (i) $C_j^A(\rho) = r_j^A + p = (4j-2)Z$ and $C_j^B(\rho) = C_j^A(\rho) + p = 4jZ$ for $j \in \mathcal{I}_1$, and (ii) $C_j^A(\rho) = C_j^B(\rho) = r_j^B + p = (4j-1)Z$ for $j \in \mathcal{I}_2$.

Since ρ is a normal schedule for Ψ, we have

$$\sum_{j=1}^g w_j^A C_j^A(\rho) = \sum_{j\in\mathcal{I}_1} z_j(4j-2)Z + \sum_{j\in\mathcal{I}_2} z_j(4j-1)Z = \sum_{j=1}^g z_j(4j-2)Z + \sum_{j\in\mathcal{I}_2} z_j Z \le Q_A \quad (4)$$

and

$$\sum_{j=1}^g w_j^B C_j^B(\rho) = \sum_{j\in\mathcal{I}_1} z_j(4jZ) + \sum_{j\in\mathcal{I}_2} z_j(4j-1)Z = \sum_{j=1}^g z_j(4j-1)Z + \sum_{j\in\mathcal{I}_1} z_j Z \le Q_B. \quad (5)$$

Recall that $Q_A = \sum_{j=1}^g z_j(4j-2)Z + Z^2$ and $Q_B = \sum_{j=1}^g z_j(4j-1)Z + Z^2$. From (4) and (5), we deduce that $\sum_{j\in\mathcal{I}_1} z_j \le Z$ and $\sum_{j\in\mathcal{I}_2} z_j \le Z$. Since $\sum_{j\in\mathcal{I}_1} z_j + \sum_{j\in\mathcal{I}_2} z_j = \sum_{j=1}^g z_j = 2Z$, we further deduce that $\sum_{j\in\mathcal{I}_1} z_j = \sum_{j\in\mathcal{I}_2} z_j = Z$. Therefore, $(\mathcal{I}_1, \mathcal{I}_2)$ is a solution to Φ. □

Combining the results of Lemmas 4.2 and 4.3, we obtain the following theorem.

Theorem 4.1 *Problem $1|\beta^*, \sum w_j^A C_j^A \le Q_A, \sum w_j^B C_j^B \le Q_B|-$ is $\mathcal{NP}$-complete.*

4.2 $\mathcal{NP}$-completeness of $1|\beta^*, \sum w_j^A U_j^A \le Q_A, \sum w_j^B U_j^B \le Q_B|-$

Given an instance $\Phi = (z_1, z_2, \ldots, z_g; Z)$ of **PP**, we build an instance Ψ of $1|\beta^*, \sum w_j^A U_j^A \le Q_A, \sum w_j^B U_j^B \le Q_B|-$ as follows:

- Each group $\mathcal{J}_X$ consists of g jobs $J_1^X, J_2^X, \ldots, J_g^X$, all with equal processing times $p = Z$, where $X = A, B$.
- Each job J_j^A, $j = 1, 2, \ldots, g$, has a release date $r_j^A = (j-1)(Z+1)$, a due date $d_j^A = j(Z+1) - 1$, and a weight $w_j^A = z_j$.
- Each job J_j^B, $j = 1, 2, \ldots, g$, has a release date $r_j^B = (j-1)(Z+1) + 1$, a due date $d_j^B = j(Z+1)$, and a weight $w_j^B = z_j$.
- The upper bounds of the two agents are $Q_A = Z$ and $Q_B = Z$, respectively.
- The decision is to find whether there is a schedule ρ (called *normal*) with $\sum_{j=1}^g w_j^A U_j^A(\rho) \le Q_A$ and $\sum_{j=1}^g w_j^B U_j^B(\rho) \le Q_B$.

Lemma 4.4 *If Φ has a solution, then Ψ has a normal schedule.*

Proof. Let $(\mathcal{I}_1, \mathcal{I}_2)$ be a solution to Φ. For each $j \in \mathcal{I}_1$, define $\mathcal{B}_j = \{J_j^A\}$. For each $j \in \mathcal{I}_2$, define $\mathcal{B}_j = \{J_j^B\}$. Define an extra batch $\mathcal{B}_{g+1} = \{J_j^A : j \in \mathcal{I}_2\} \cup \{J_j^B : j \in \mathcal{I}_1\}$. A schedule ρ for Ψ is defined by the batch processing sequence $\mathcal{BPS} = (\mathcal{B}_1, \mathcal{B}_2, \ldots, \mathcal{B}_{g+1})$.

By Lemma 2.1, we obtain that (i) $C_j^A(\rho) = j(Z+1) - 1 = d_j^A$ for $j \in \mathcal{I}_1$, (ii) $C_j^B(\rho) = j(Z+1) = d_j^B$ for $j \in \mathcal{I}_2$, and (iii) $C_j^X(\rho) = C(\mathcal{B}_{g+1}) = C(\mathcal{B}_g) + p \ge (g-1)(Z+1) + 2p > d_g^X \ge d_j^X$ for $J_j^X \in \mathcal{B}_{g+1}$. This shows that (i) $U_j^A(\rho) = 0$ and $U_j^B(\rho) = 1$ for $j \in \mathcal{I}_1$, and (ii) $U_j^A(\rho) = 1$ and $U_j^B(\rho) = 0$ for $j \in \mathcal{I}_2$.

Since $\sum_{j \in \mathcal{I}_1} z_j = \sum_{j \in \mathcal{I}_2} z_j = Z$, it follows that $\sum_{j=1}^g w_j^A U_j^A(\rho) = \sum_{j \in \mathcal{I}_2} w_j^A = \sum_{j \in \mathcal{I}_2} z_j = Z = Q_A$ and $\sum_{j=1}^g w_j^B U_j^B(\rho) = \sum_{j \in \mathcal{I}_1} w_j^B = \sum_{j \in \mathcal{I}_1} z_j = Q_B$. Therefore, ρ is a normal schedule for Ψ. □

Lemma 4.5 *If Ψ has a normal schedule, then Φ has a solution.*

Proof. Let ρ be a normal schedule for Ψ. From the definition of job parameters, we have

$$r_1^A < r_1^B < d_1^A < d_1^B = r_2^A < r_2^B < d_2^A < d_2^B = r_3^A < \cdots = r_g^A < r_g^B < d_g^A < d_g^B \tag{6}$$

and

$$d_j^X = r_j^X + p,\ X \in \{A, B\},\ j \in \{1, 2, \ldots, g\}. \tag{7}$$

From (6) and (7), it follows that $C_j^X(\rho) = d_j^X$ if job J_j^X is early in ρ for $X \in \{A, B\}$. Define $\mathcal{I}_1 = \{j : C_j^A(\rho) = d_j^A\}$ and $\mathcal{I}_2 = \{j : C_j^B(\rho) = d_j^B\}$. This implies that all jobs J_j^A ($j \in \mathcal{I} \setminus \mathcal{I}_1$) and J_j^B ($j \in \mathcal{I} \setminus \mathcal{I}_2$) in ρ are tardy.

Since ρ is a normal schedule for Ψ, we have

$$\sum_{j=1}^{g} w_j^A U_j^A(\rho) = \sum_{j=1}^{g} w_j^A - \sum_{j\in\mathcal{I}_1} w_j^A = \sum_{j=1}^{g} z_j - \sum_{j\in\mathcal{I}_1} z_j = 2Z - \sum_{j\in\mathcal{I}_1} z_j \le Q_A \tag{8}$$

and

$$\sum_{j=1}^{g} w_j^B U_j^B(\rho) = \sum_{j=1}^{g} w_j^B - \sum_{j\in\mathcal{I}_2} w_j^B = \sum_{j=1}^{g} z_j - \sum_{j\in\mathcal{I}_2} z_j = 2Z - \sum_{j\in\mathcal{I}_2} z_j \le Q_B. \tag{9}$$

Recall that $Q_A = Q_B = Z$. From (8) and (9), we deduce that

$$\sum_{j\in\mathcal{I}_1} z_j \ge Z \text{ and } \sum_{j\in\mathcal{I}_2} z_j \ge Z. \tag{10}$$

Based on the facts that $\mathcal{I}_1 \subseteq \mathcal{I}$, $\mathcal{I}_2 \subseteq \mathcal{I}$ and $\mathcal{I}_1 \cap \mathcal{I}_2 = \emptyset$, we have $(\mathcal{I}_1 \cup \mathcal{I}_2) \subseteq \mathcal{I}$. Hence,

$$\sum_{j\in\mathcal{I}_1} z_j + \sum_{j\in\mathcal{I}_2} z_j \le \sum_{j\in\mathcal{I}} z_j = 2Z. \tag{11}$$

From (10) and (11), we deduce that $\sum_{j\in\mathcal{I}_1} z_j = \sum_{j\in\mathcal{I}_2} z_j = Z$ and $\mathcal{I}_1 \cup \mathcal{I}_2 = \mathcal{I}$. Therefore, $(\mathcal{I}_1, \mathcal{I}_2)$ is a solution to Φ. □

Combining the results of Lemmas 4.4 and 4.5, we obtain the following theorem.

Theorem 4.2 *Problem* $1|\beta^*, \sum w_j^A U_j^A \le Q_A, \sum w_j^B U_j^B \le Q_B|-$ *is* $\mathcal{NP}$*-complete.*

4.3 $\mathcal{NP}$-completeness of $1|\beta^*, \sum Y_j^A \le Q_A, \sum Y_j^B \le Q_B|-$

Given an instance $\Phi = (z_1, z_2, \ldots, z_g; Z)$ of **PP**, we build an instance Ψ of $1|\beta^*, \sum Y_j^A \le Q_A, \sum Y_j^B \le Q_B|-$ as follows:

- Each group $\mathcal{J}_X$ consists of g jobs $J_1^X, J_2^X, \ldots, J_g^X$, all with equal processing times $p = 2Z$, where $X = A, B$.
- Each job J_j^A, $j = 1, 2, \ldots, g$, has a release date $r_j^A = (4j-4)Z$ and a due date $d_j^A = (4j-2)Z$.
- Each job J_j^B, $j = 1, 2, \ldots, g$, has a release date $r_j^B = (4j-4)Z + z_j$ and a due date $d_j^B = 4jZ - z_j$.
- The upper bounds of the two agents are $Q_A = Z$ and $Q_B = Z$, respectively.
- The decision is to find whether there is a schedule ρ (called *normal*) with $\sum_{j=1}^{g} Y_j^A(\rho) \le Q_A$ and $\sum_{j=1}^{g} Y_j^B(\rho) \le Q_B$.

Lemma 4.6 *If* Φ *has a solution, then* Ψ *has a normal schedule.*

Proof. Let $(\mathcal{I}_1, \mathcal{I}_2)$ be a solution to Φ. For each $j \in \mathcal{I}_1$, define $\mathcal{B}_j^1 = \{J_j^A\}$ and $\mathcal{B}_j^2 = \{J_j^B\}$. For each $j \in \mathcal{I}_2$, define $\mathcal{B}_j^1 = \{J_j^A, J_j^B\}$ and $\mathcal{B}_j^2 = \emptyset$. A schedule ρ for Ψ is defined by the batch processing sequence $\mathcal{BPS} = (\mathcal{B}_1^1, \mathcal{B}_1^2, \mathcal{B}_2^1, \mathcal{B}_2^2, \ldots, \mathcal{B}_g^1, \mathcal{B}_g^2)$.

By Lemma 2.1, we obtain that (i) $C_j^A(\rho) = r_j^A + p = (4j-2)Z = d_j^A$ and $C_j^B(\rho) = C_j^A(\rho) + p = 4jZ > 4jZ - z_j = d_j^B$ for $j \in \mathcal{I}_1$, and (ii) $C_j^A(\rho) = r_j^B + p = (4j-2)Z + z_j > d_j^A$ and $C_j^B(\rho) = C_j^A(\rho) = (4j-2)Z + z_j < 4jZ - z_j = d_j^B$ for $j \in \mathcal{I}_2$. This implies that (i) $Y_j^A(\rho) = 0$ and $Y_j^B(\rho) = z_j$ for $j \in \mathcal{I}_1$, and (ii) $Y_j^A(\rho) = z_j$ and $Y_j^B(\rho) = 0$ for $j \in \mathcal{I}_2$.

Since $\sum_{j\in\mathcal{I}_1} z_j = \sum_{j\in\mathcal{I}_2} z_j = Z$, it follows that $\sum_{j=1}^g Y_j^A(\rho) = \sum_{j\in\mathcal{I}_2} Y_j^A(\rho) = \sum_{j\in\mathcal{I}_2} z_j = Z = Q_A$ and $\sum_{j=1}^g Y_j^B(\rho) = \sum_{j\in\mathcal{I}_1} Y_j^B(\rho) = \sum_{j\in\mathcal{I}_1} z_j = Z = Q_B$. Therefore, ρ is a normal schedule for Ψ. □

Lemma 4.7 *If Ψ has a normal schedule, then Φ has a solution.*

Proof. Let ρ be a normal ERD-batch schedule for Ψ (Lemma 2.2). Based on Lemma 4.1 and the fact that $p = 2Z > Q_A = Q_B$, it follows that each job J_j^X is non-late in ρ for $X = A, B$ and $j = 1, 2, \ldots, n_X$. From the definition of job parameters, we have

$$r_j^A < r_j^A + z_j = r_j^B < r_j^A + p = d_j^A < d_j^B = r_j^B + p < d_j^A + p = r_{j+1}^A,\ j \in \{1, 2, \ldots, g\}, \quad (12)$$

where $r_{g+1}^A = 4gZ$. In view of Lemmas 2.1, 2.2, and (12), the starting time of J_j^A in ρ is constrained to be either r_j^A or r_j^B. In the former case, J_j^B starts at time $r_j^A + p$. In the latter case, J_j^B starts at time r_j^B as well. Define $\mathcal{I}_1 = \{j : S_j^A(\rho) = r_j^A\}$ and $\mathcal{I}_2 = \{j : S_j^A(\rho) = r_j^B\}$. By simple computation, we obtain that (i) $C_j^A(\rho) = r_j^A + p = (4j-2)Z = d_j^A$ and $C_j^B(\rho) = r_j^A + 2p = 4jZ > 4jZ - z_j = d_j^B$ for $j \in \mathcal{I}_1$, and (ii) $d_j^A = (4j-2)Z < (4j-2)Z + z_j = C_j^A(\rho) = C_j^B(\rho) = r_j^B + p < d_j^B = 4jZ - z_j$ for $j \in \mathcal{I}_2$. This implies that (i) $Y_j^A(\rho) = 0$ and $Y_j^B(\rho) = z_j$ for $j \in \mathcal{I}_1$, and (ii) $Y_j^A(\rho) = z_j$ and $Y_j^B(\rho) = 0$ for $j \in \mathcal{I}_2$.

Since ρ is a normal schedule for Ψ, we have

$$\sum_{j=1}^{g} Y_j^A(\rho) = \sum_{j\in\mathcal{I}_2} Y_j^A(\rho) = \sum_{j\in\mathcal{I}_2} z_j \le Q_A \quad (13)$$

and

$$\sum_{j=1}^{g} Y_j^B(\rho) = \sum_{j\in\mathcal{I}_1} w_j^B = \sum_{j\in\mathcal{I}_1} z_j \le Q_B. \quad (14)$$

Recall that $Q_A = Q_B = Z$. From (13) and (14), we deduce that $\sum_{j\in\mathcal{I}_1} z_j \le Z$ and $\sum_{j\in\mathcal{I}_2} z_j \le Z$. Since $\sum_{j\in\mathcal{I}_1} z_j + \sum_{j\in\mathcal{I}_2} z_j = \sum_{j=1}^g z_j = 2Z$, we further deduce that $\sum_{j\in\mathcal{I}_1} z_j = \sum_{j\in\mathcal{I}_2} z_j = Z$. Therefore, $(\mathcal{I}_1, \mathcal{I}_2)$ is a solution to Φ. □

Combining the results of Lemmas 4.6 and 4.7, we obtain the following theorem.

Theorem 4.3 *Problem* $1|\beta^*, \sum Y_j^A \leq Q_A, \sum Y_j^B \leq Q_B|-$ *is* $\mathcal{NP}$*-complete.*

Based on Lemma 4.1 and the proof of Theorem 4.3, the following two corollaries hold.

Corollary 4.1 *Problem* $1|\beta^*, \sum T_j^A \leq Q_A, \sum Y_j^B \leq Q_B|-$ *is* $\mathcal{NP}$*-complete.*

Corollary 4.2 *Problem* $1|\beta^*, \sum T_j^A \leq Q_A, \sum T_j^B \leq Q_B|-$ *is* $\mathcal{NP}$*-complete.*

4.4 $\mathcal{NP}$-completeness of $1|\beta^*, \sum C_j^A \leq Q_A, \sum w_j^B U_j^B \leq Q_B|-$

Theorem 4.4 *Problem* $1|\beta^*, \sum C_j^A \leq Q_A, \sum w_j^B U_j^B \leq Q_B|-$ *is* $\mathcal{NP}$*-complete.*

Proof. Given an instance $\Phi = (z_1, z_2, \ldots, z_g; Z)$ of **PP**, we build an instance Ψ of $1|\beta^*, \sum C_j^A \leq Q_A, \sum w_j^B U_j^B \leq Q_B|-$ as follows: (i) for $X \in \{A, B\}$, $\mathcal{J}_X = \{J_1^X, J_2^X, \ldots, J_g^X\}$, $p = 2Z$; (ii) for $j = 1, 2, \ldots, g$, $r_j^A = (4j-4)Z$, $r_j^B = (4j-4)Z + z_j$, and $w_j^B = z_j$; (iii) $Q_A = (2g^2+1)Z$ and $Q_B = Z$. The proof follows an approach analogous to that used in Theorems 4.1 and 4.2. □

4.5 $\mathcal{NP}$-completeness of $1|\beta^*, \sum Y_j^A \leq Q_A, \sum w_j^B U_j^B \leq Q_B|-$

Theorem 4.5 *Problem* $1|\beta^*, \sum Y_j^A \leq Q_A, \sum w_j^B U_j^B \leq Q_B|-$ *is* $\mathcal{NP}$*-complete.*

Proof. Given an instance $\Phi = (z_1, z_2, \ldots, z_g; Z)$ of **PP**, we build an instance Ψ of $1|\beta^*, \sum Y_j^A \leq Q_A, \sum w_j^B U_j^B \leq Q_B|-$ as follows: (i) for $X \in \{A, B\}$, $\mathcal{J}_X = \{J_1^X, J_2^X, \ldots, J_g^X\}$, $p = 2Z$; (ii) for $j = 1, 2, \ldots, g$, $r_j^A = (4j-4)Z$, $d_j^A = (4j-2)Z$, $r_j^B = (4j-4)Z + z_j$, $d_j^B = (4j-2)Z + z_j$, and $w_j^B = z_j$; (iii) $Q_A = Z$ and $Q_B = Z$. The proof follows an approach analogous to that used in Theorems 4.2 and 4.3. □

Based on Lemma 4.1 and the proof of Theorem 4.5, the following corollary hold.

Corollary 4.3 *Problem* $1|\beta^*, \sum T_j^A \leq Q_A, \sum w_j^B U_j^B \leq Q_B|-$ *is* $\mathcal{NP}$*-complete.*

4.6 $\mathcal{NP}$-completeness of $1|\beta^*, \sum C_j^A \leq Q_A, \sum Y_j^B \leq Q_B|-$

Theorem 4.6 *Problem* $1|\beta^*, \sum C_j^A \leq Q_A, \sum Y_j^B \leq Q_B|-$ *is* $\mathcal{NP}$*-complete.*

Proof. Given an instance $\Phi = (z_1, z_2, \ldots, z_g; Z)$ of **PP**, we build an instance of $1|\beta^*, \sum C_j^A \leq Q_A, \sum Y_j^B \leq Q_B|-$ as follows: (i) for $X \in \{A, B\}$, $\mathcal{J}_X = \{J_1^X, J_2^X, \ldots, J_g^X\}$, $p = 2Z$; (ii) for $j = 1, 2, \ldots, g$, $r_j^A = (4j-4)Z$, $r_j^B = (4j-4)Z + z_j$, and $d_j^B = 4jZ - z_j$; (iii) $Q_A = (2g^2+1)Z$ and $Q_B = Z$. The proof follows an approach analogous to that used

in Theorems 4.1 and 4.3. □

Based on Lemma 4.1 and the proof of Theorem 4.6, the following corollary hold.

Corollary 4.4 *Problem* $1|\beta^*, \sum C_j^A \le Q_A, \sum T_j^B \le Q_B|-$ *is* $\mathcal{NP}$*-complete.*

5 An exact DP algorithm for $1|\beta^*|\#(f_{\max}^A, f_{\max}^B)$

Given the intractability of problem $1|\beta^*|\#(f_{\max}^A, f_{\max}^B)$, we develop a pseudo-polynomial-time DP framework for its exact solution. In view of Lemmas 2.1 and 2.2, we focus exclusively on the ERD-batch schedules, where the completion times of all batches belong to $\mathcal{R}$, as defined by (2). The remaining decision to be made is how to partition jobs into bathes. For ease of notation, define $\mathcal{J}_j = \{J_1, J_2, \ldots, J_j\}$ for $j = 1, 2, \ldots, n$ and $\mathcal{J}_0 = \emptyset$.

In our DP framework, we sequentially scan jobs in $\mathcal{J}$ according to the ordering defined by (1), dynamically constructing the state space and applying a dominance rule to improve computational efficiency. For each $j \in \{1, 2, \ldots, n\}$, let Φ_j represent the set of all schedules ρ for $\mathcal{J}_j$, where ρ is a partial schedule derived from some ERD-batch schedule ρ' for $\mathcal{J}$. This partial schedule ρ can be constructed from ρ' by removing jobs $J_{j+1}, J_{j+2}, \ldots, J_n$. Each schedule $\rho \in \Phi_j$ can be uniquely characterized by a state $\mathcal{S}(\rho) = (j, t, u, v)$, where (i) t represents the last job's completion time; (ii) u denotes agent A's objective value; and (iii) v indicates agent B's objective value. In this framework, ρ is classified as an ERD-batch schedule that attains the state (j, t, u, v), i.e., $t = C_{\max}(\rho) = C_j(\rho)$, $u = \sum_{J_k \in \mathcal{J}_j \cap \mathcal{J}^A} f_k^A(\rho)$, and $v = \sum_{J_k \in \mathcal{J}_j \cap \mathcal{J}^B} f_k^B(\rho)$.

Let $\mathcal{H}_j = \{\mathcal{S}(\rho) : \rho \in \Phi_j\}$ denote the state space encompassing all attainable states derived from the schedules in Φ_j. The initial set $\mathcal{H}_0$ consists solely of the state $\mathcal{S}(\rho_0) = \{(0, 0, 0, 0)\}$, where ρ_0 denotes an empty schedule. For each index $j \in \{1, 2, \ldots, n\}$, $\mathcal{H}_j$ can be generated from $\mathcal{H}_i$ for $i = 0, 1, \ldots, j-1$. For a given state $(j, t, u, v) \in \mathcal{H}_j$, let $\rho_j \in \Phi_j$ be a feasible ERD-batch schedule that attains the state $\mathcal{S}(\rho_j) = (j, t, u, v)$. According to Lemma 2.2, there exists an index $i \in \{0, 1, \ldots, j-1\}$ such that the last batch in ρ_j consists of jobs $J_{i+1}, J_{i+2}, \ldots, J_j$. Let ρ_i be the schedule for $\mathcal{J}_i$ constructed from ρ_j by removing jobs $J_{i+1}, J_{i+2}, \ldots, J_j$. This implies that $\rho_i \in \Phi_i$. Assume $\mathcal{S}(\rho_i) = (i, t', u', v') \in \mathcal{H}_i$. Then in view of Lemmas 2.1 and 2.2, it follows that $t' = C_i(\rho_i) = C_i(\rho_j) \in \mathcal{R}$ and $\max\{t', r_j\} + p = t = C_{i+1}(\rho_j) = C_{i+2}(\rho_j) = \cdots = C_j(\rho_j)$. In this context, we obtain that $u = u' + \sum_{J_k \in (\mathcal{J}_j \setminus \mathcal{J}_i) \cap \mathcal{J}^A} f_k^A(t)$ and $v = v' + \sum_{J_k \in (\mathcal{J}_j \setminus \mathcal{J}_i) \cap \mathcal{J}^B} f_k^B(t)$.

The above discussion reveals that set $\mathcal{H}_j$ can be generated from sets $\mathcal{H}_i$ for $i = 0, 1, \ldots, j-1$. To streamline the state space of the DP algorithm, the following dominance rule is established.

Lemma 5.1 *Given any two states (j,t,u_1,v_1) and (j,t,u_2,v_2) in $\mathcal{H}_j$ satisfying $u_1 \leq u_2$ and $v_1 \leq v_2$, the latter state can be removed from $\mathcal{H}_j$.*

Proof. Let ρ_j^1 and ρ_j^2 be two ERD-batch schedules that attain the states (j,t,u_1,v_1) and (j,t,u_2,v_2), respectively. Let $\mathcal{BPS} = (\mathcal{B}_1, \mathcal{B}_2, \ldots, \mathcal{B}_k)$ denote a batch processing sequence of $\mathcal{J} \setminus \mathcal{J}_j$. Appending $\mathcal{BPS}$ to ρ_j^2 yields a complete schedule ρ_n^2 for $\mathcal{J}$. Similarly, appending $\mathcal{BPS}$ to ρ_j^1 yields a complete schedule ρ_n^1 for $\mathcal{J}$. Since the contribution of jobs in $\mathcal{J} \setminus \mathcal{J}_j$ is identical for both ρ_n^1 and ρ_n^2, we have $f_{\rm sum}^A(\rho_n^1) \leq f_{\rm sum}^A(\rho_n^2)$ and $f_{\rm sum}^B(\rho_n^1) \leq f_{\rm sum}^B(\rho_n^2)$. Therefore, we conclude that ρ_j^1 dominates ρ_j^2. □

For each $j = 1, 2, \ldots, n$ and $t \in \mathcal{R}$, define

$$g_j(t) = \begin{cases} f_j^A(t), & \text{if } J_j \in \mathcal{J}^A; \\ 0, & \text{if } J_j \in \mathcal{J}^B. \end{cases} \tag{15}$$

and

$$h_j(t) = \begin{cases} f_j^B(t), & \text{if } J_j \in \mathcal{J}^B; \\ 0, & \text{if } J_j \in \mathcal{J}^A. \end{cases} \tag{16}$$

Recall that $\Lambda = r_{\max}(\mathcal{J}) + 2p - 1$. In view of the above discussion, we introduce the following generic DP algorithm to solve $1|\beta^*|\#(f_{\rm sum}^A, f_{\rm sum}^B)$.

Algorithm DP$_1$. An exact algorithm for $1|\beta^*|\#(f_{\rm sum}^A, f_{\rm sum}^B)$

Input: $\mathcal{J}^X = \{J_1^X, J_2^X, \ldots, J_{n_X}^X\}$, p, r_j^X, w_j^X, d_j^X, f_j^X for $X = A, B$ and $j = 1, 2, \ldots, n_X$.

Step 1. (**Pre-processing**) Renumber the jobs in $\mathcal{J}$ in line with (1) and determine the set $\mathcal{R}$ in line with (2). For each $j = 1, 2, \ldots, n$ and $t \in \mathcal{R}$, if $t \geq r_j + p$, then compute $G_j(t) = \sum_{k=1}^{j} g_k(t)$ and $H_j(t) = \sum_{k=1}^{j} h_k(t)$, where $G_0(t) = H_0(t) = 0$.

Step 2. (**Initialization**) Set $\mathcal{H}_0 = \{(0,0,0,0)\}$, $\mathcal{H}_j = \emptyset$, and $\mathcal{H}_{j,i} = \emptyset$ for $j = 1, 2, \ldots, n$ and $i = 0, 1, \ldots, j-1$.

Step 3. (**Generation**) Generate $\mathcal{H}_j$ from $\mathcal{H}_0, \mathcal{H}_1, \ldots, \mathcal{H}_{j-1}$.

For $j = 1$ to n **do**

For $i = 0$ to $j - 1$ **do**

For each $(i,t,u,v) \in \mathcal{H}_i$ **do**

Set $t' = \max\{t, r_j\} + p$, $u' = u + G_j(t') - G_i(t')$, and $v' = v + H_j(t') - H_i(t')$.

If $\max\{t, r_j\} + p \leq \Lambda$, **then**

Add (j, t', u', v') to $\mathcal{H}_{j,i}$.

End if

End for

End for

Set $\mathcal{H}_j = \mathcal{H}_{j,0} \cup \mathcal{H}_{j,1} \cup \cdots \cup \mathcal{H}_{j,j-1}$.

(**Elimination**) For any two states (j,t,u,v) and (j,t,u',v') in $\mathcal{H}_j$ such that $u \leq u'$ and $v \leq v'$, retain only the first one in $\mathcal{H}_j$.

End for

Step 4. (**Extraction**) For any two states (n,t,u,v) and (n,t',u',v') in $\mathcal{H}_n$ such that $u \leq u'$ and $v \leq v'$, retain only the first one in $\mathcal{H}_n$.

Step 5. (**Result**) Set $\mathcal{Q} = \{(u,v) : (n,t,u,v) \in \mathcal{H}_n\}$.

Output: The Pareto-optimal points in $\mathcal{Q}$ and their associated Pareto-optimal schedules.

Define $\Delta_A = \sum_{j=1}^{n_A} f_j^A(\Lambda)$, $\Delta_B = \sum_{j=1}^{n_B} f_j^B(\Lambda)$, and $\Delta = \min\{\Delta_A, \Delta_B\}$. Clearly, Δ_X is a valid upper bound on $f_{\max}^X$ of $1|\beta^*|\#(f_{\max}^A, f_{\max}^B)$, where $X = A, B$.

Theorem 5.1 *Algorithm* $\mathbf{DP_1}$ *solves* $1|\beta^*|\#(f_{\max}^A, f_{\max}^B)$ *in* $O(n^4\Delta)$ *time.*

Proof. Based on Lemmas 2.1, 2.2, 5.1, and the preceding analysis, $\mathbf{DP_1}$ provides a correct solution to $1|\beta^*|\#(f_{\max}^A, f_{\max}^B)$. In Step 1, the sorting action requires $O(n \log n)$ time and the computation of all $G_j(t)$ and $H_j(t)$ takes $O(n^3)$ time. Step 2 requires linear time for [Initialization]. For each state $(i,t,u,v) \in \mathcal{H}_i$, its components satisfy $i \in \{0,1,\ldots,j-1\}$, $t \in \mathcal{R}$ with $|\mathcal{R}| = O(n^2)$, $u \leq \Delta_A$, and $v \leq \Delta_B$. Hence, after executing [Elimination] in Step 3, the number of distinct states in $\mathcal{H}_i$ is at most $O(n^2\Delta)$. Hence, each set $\mathcal{H}_{j,i}$ can be generated from $\mathcal{H}_i$ in $O(n^2\Delta)$ time. This further implies that the generation of $\mathcal{H}_j$ takes $O(n^3\Delta)$ time. Step 4 requires $O(n^2\Delta)$ time for [Extraction], while Step 5 requires $O(\Delta)$ time for [Result]. Since Step 3 executes n iterations, the time complexity of $\mathbf{DP_1}$ is bounded by $O(n^4\Delta)$. □

For each $j = 1,2,\ldots,n$, storing all values of $G_j(t)$ and $H_j(t)$ consumes $O(n^2)$ space. According to Lemma 5.1, the space required for states $(j,t,u,v) \in \mathcal{H}_j$ is bounded by $O(n^2\Delta)$. Therefore, the total space complexity of $\mathbf{DP_1}$ is $O(n^3\Delta)$.

Note that if $\gamma^X = \sum w_j^X U_j^X$, $X \in \{A,B\}$, then $\Delta_X \leq \sum_{j=1}^{n_X} w_j^X$. As a direct consequence of Theorem 5.1, we obtain the following two corollaries.

Corollary 5.1 *Algorithm* $\mathbf{DP_1}$ *solves* $1|\beta^*|\#(f_{\max}^A, \sum w_j^B U_j^B)$ *in* $O(n^4 \sum_{j=1}^{n_B} w_j^B)$ *time.*

Corollary 5.2 *Algorithm* $\mathbf{DP_1}$ *solves* $1|\beta^*|\#(f_{\max}^A, \sum U_j^B)$ *in* $O(n^4 n_B)$ *time.*

6 Algorithms for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$

In this section, we study problem $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$. Since Lemma 2.4 shows that the single-agent problem $1|\beta^*|f_{\max}$ is solvable in $O(n^3)$ time, we can assume that $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$ is feasible. Let ρ^* be an optimal schedule of this problem, and let F^* be its associated objective value, i.e., $F^* = f^A_{\max}(\rho^*) = \sum_{j=1}^{n_A} f^A_j(\rho^*)$. In this context, we have $f^B_{\max}(\rho^*) \leq Q_B$.

Definition 6.1 *An algorithm $\mathcal{H}$ for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$ is a λ-approximation algorithm if it delivers a schedule ρ with $f^A_{\max}(\rho) \leq \lambda F^*$ and $f^B_{\max}(\rho) \leq Q_B$. A family of algorithms $\{H_\epsilon : \epsilon > 0\}$ is an FPTAS if, for each $\epsilon > 0$, $\mathcal{H}_\epsilon$ is a $(1+\epsilon)$-approximation algorithm that runs in polynomial time in the input length and $1/\epsilon$.*

Definition 6.2 *An algorithm $\mathcal{H}$ for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$ is a super-dual λ-approximation algorithm if it delivers a schedule ρ with $f^A_{\max}(\rho) \leq F^*$ and $f^B_{\max}(\rho) \leq \lambda Q_B$. A family of algorithms $\{\mathcal{H}_\epsilon : \epsilon > 0\}$ is a super-dual FPTAS if, for each $\epsilon > 0$, H_ϵ is a super-dual $(1+\epsilon)$-approximation algorithm that runs in polynomial time in the input length and $1/\epsilon$.*

6.1 An exact algorithm for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$

The exact algorithm $\mathbf{DP_1}$ described in Section 5 for $1|\beta^*|\#(f^A_{\max}, f^B_{\max})$ can be directly applied to solve $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$ in $O(n^4\Delta)$ time. However, to facilitate the development of an FPTAS for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$, we propose a refined and modified DP formulation.

Algorithm $\mathbf{DP_2}(\mathbf{Q_B})$. An exact algorithm for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$

Input: $\mathcal{J}^X = \{J^X_1, J^X_2, \ldots, J^X_{n_X}\}$, p, Q_B, r^X_j, w^X_j, d^X_j, f^X_j for $X = A, B$ and $j = 1, 2, \ldots, n_X$.

Step 1. (**Pre-processing**) Same as that in Step 1 of Algorithm $\mathbf{DP_1}$.

Step 2. (**Initialization**) Set $\mathcal{D}_0 = \{(0,0,0,0)\}$, $\mathcal{D}_j = \emptyset$, and $\mathcal{D}_{j,i} = \emptyset$ for $j = 1, 2, \ldots, n$ and $i = 0, 1, \ldots, j-1$.

Step 3. (**Generation**) Generate $\mathcal{D}_j$ from $\mathcal{D}_0, \mathcal{D}_1, \ldots, \mathcal{D}_{j-1}$.

For $j = 1$ to n **do**

For $i = 0$ to $j-1$ **do**

For each $(i, t, u, v) \in \mathcal{D}_i$ **do**

Set $t' = \max\{t, r_j\} + p$, $u' = u + G_j(t') - G_i(t')$, $v' = v + H_j(t') - H_i(t')$.

If $t' \leq \Lambda$ and $v' \leq Q_B$, **then**

Add (j, t', u', v') to $\mathcal{D}_{j,i}$.

End if

End for

End for

Set $\mathcal{D}_j = \mathcal{D}_{j,0} \cup \mathcal{D}_{j,1} \cup \cdots \cup \mathcal{D}_{j,j-1}$.

(**Elimination**) For any two states (j,t,u,v) and (j,t,u,v') in $\mathcal{D}_j$ such that $v \leq v'$, retain only the first one in $\mathcal{D}_j$.

End for

Step 4. (**Result**) Set $F^* = \min\{u : (n,t,u,v) \in \mathcal{D}_n\}$.

Output: The optimal solution value F^* and its associated optimal schedule ρ^*.

According to the [Elimination] procedure in Step 3, at most $O(n^2 Q_B)$ states are retained in $\mathcal{D}_j$. Similar to analysis of Theorem 5.1, we obtain the following theorem.

Theorem 6.1 *Algorithm* $\mathbf{DP_2(Q_B)}$ *solves* $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$ *in* $O(n^4 Q_B)$ *time.*

6.2 An FPTAS for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$

We first employ the *interval-partitioning* technique introduced by Sahni (1976) to design a generic approximation procedure for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$. Subsequently, we apply the *bound improvement procedure* (BIP) proposed by Chubanov et al. (2006) to establish valid lower and upper bounds. Finally, we construct an FPTAS based on these components.

To design a generic approximation procedure, let $F_{\mathcal{LB}}$ and $F_{\mathcal{UB}}$ be two given lower and upper bounds that satisfy $0 < F_{\mathcal{LB}} \leq F^* \leq F_{\mathcal{UB}}$.

Algorithm $\mathbf{DP_{gen}(Q_B)}$**.** A generic approximation procedure for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$

Input: $\mathcal{J}^X = \{J^X_1, J^X_2, \ldots, J^X_{n_X}\}$, p, Q_B, ϵ, $F_{\mathcal{LB}}$, $F_{\mathcal{UB}}$, r^X_j, w^X_j, d^X_j, f^X_j for $X = A, B$ and $j = 1, 2, \ldots, n_X$.

Step 1. (**Partition**) Set $\Theta = F_{\mathcal{UB}} + \epsilon F_{\mathcal{LB}}$, $\theta = \frac{\epsilon F_{\mathcal{LB}}}{n-1}$, and $z = \lceil \frac{\Theta}{\theta} \rceil = \lceil (n-1)(\frac{F_{\mathcal{UB}}}{\epsilon F_{\mathcal{LB}}} + 1) \rceil$. Divide the interval $[0, \Theta]$ into z subintervals $\mathcal{I}_k$ with $\mathcal{I}_k = [(k-1)\theta, k\theta)$ for $k = 1, 2, \ldots, z-1$, and $I_z = [(z-1)\theta, \Theta]$.

Step 2. (**Pre-processing**) Same as that in Step 1 of Algorithm $\mathbf{DP_1}$.

Step 3. (**Initialization**) Set $\mathcal{D}^\epsilon_0 = \{(0,0,0,0)\}$, and $\mathcal{D}^\epsilon_j = \emptyset$, and $\mathcal{D}^\epsilon_{j,i} = \emptyset$ for $j = 1, 2, \ldots, n$ and $i = 0, 1, \ldots, j-1$.

Step 4. (**Generation**) Generate $\mathcal{D}^\epsilon_j$ from $\mathcal{D}^\epsilon_0, \mathcal{D}^\epsilon_1, \ldots, \mathcal{D}^\epsilon_{j-1}$.

For $j = 1$ to n **do**

For $i = 0$ to $j-1$ **do**

For each $(i,t,u,v) \in \mathcal{D}_i^\epsilon$ **do**

Set $t' = \max\{t, r_j\} + p$, $u' = u + G_j(t') - G_i(t')$, and $v' = v + H_j(t') - H_i(t')$.

If $t' \le \Lambda$, $u' \le \Theta$, and $v' \le Q_B$, **then**

Add (j,t',u',v') to $\mathcal{D}_{j,i}^\epsilon$.

End if

End for

End for

Set $\mathcal{D}_j^\epsilon = \mathcal{D}_{j,0}^\epsilon \cup \mathcal{D}_{j,1}^\epsilon \cup \cdots \cup \mathcal{D}_{j,j-1}^\epsilon$.

(**Elimination**) Among all states $(j,t,u,v) \in \mathcal{D}_j^\epsilon$ sharing the same t value and $u \in \mathcal{I}_k$, retain only the one with the smallest v value.

End for

Step 5. (**Result**) Set $\overline{F} = \min\{u : (n,t,u,v) \in \mathcal{D}_n^\epsilon\}$.

Output: The approximate objective value $\overline{F}$ and its associated schedule $\bar{\rho}$.

Lemma 6.1 *For any eliminated state $(j,t,u,v) \in \mathcal{D}_j$ with $u \le F_{\mathcal{UB}}$, Algorithm $\mathbf{DP_{gen}(Q_B)}$ finds a state $(j,t,\bar{u},\bar{v}) \in \mathcal{D}_j^\epsilon$ with $\bar{u} \le u + (j-1)\theta$ and $\bar{v} \le v$.*

Proof. We prove the lemma via induction on $j = 1, 2, \ldots, n$. From the execution of $\mathbf{DP_2(Q_B)}$ and $\mathbf{DP_{gen}(Q_B)}$, we have $\mathcal{D}_1 = \mathcal{D}_1^\epsilon = \{(1, r_1+p, f_1^A(r_1+p), 0)\}$ if $J_1 \in \mathcal{J}^A$, and $\mathcal{D}_1 = \mathcal{D}_1^\epsilon = \{(1, r_1+p, 0, f_1^B(r_1+p))\}$ if $J_1 \in \mathcal{J}^B$. Thus, the lemma holds for $j = 1$.

Assume that the lemma is correct for all $k = 1, 2, \ldots, j-1$ with some $j \ge 2$. Then, for any eliminated state $(k,t',u',v') \in \mathcal{D}_k$ with $k \le j-1$ and $u' \le F_{\mathcal{UB}}$, there exists a state $(k,t',\bar{u}',\bar{v}') \in \mathcal{D}_k^\epsilon$ with $\bar{u}' \le u' + (k-1)\theta$ and $\bar{v}' \le v'$.

We now prove that the lemma is correct for $k = j$. Consider an arbitrary eliminated state $(j,t,u,v) \in \mathcal{D}_j$ with $u \le F_{\mathcal{UB}}$. Let ρ_j be a feasible ERD-batch schedule for $\mathcal{J}_j$ attaining the state $\mathcal{S}(\rho_j) = (j,t,u,v)$. Since $D_j = \mathcal{D}_{j,0} \cup \mathcal{D}_{j,1} \cup \cdots \cup \mathcal{D}_{j,j-1}$, we can assume that $(j,t,u,v) \in D_{j,i}$ for some $i = 0, 1, \ldots, j-1$. In view of Lemma 2.2, the last batch consists of jobs $J_{i+1}, J_{i+2}, \ldots, J_j$ in ρ_j. Let ρ_i be the schedule for $\mathcal{J}_i$ constructed from ρ_j by removing its last batch. Write $\mathcal{S}(\rho_i) = (i, \tilde{t}, \tilde{u}, \tilde{v})$. Since $(i,\tilde{t},\tilde{u},\tilde{v}) \in \mathcal{D}_i$ and according to the execution of $\mathbf{DP_2(Q_B)}$, we have $\tilde{t} = C_i(\rho_i) = C_i(\rho_j)$, $t = \max\{\tilde{t}, r_j\} + p = C_{i+1}(\rho_j) = C_{i+2}(\rho_j) = \cdots = C_j(\rho_j)$, $u = \tilde{u} + G_j(t) - G_i(t)$, and $v = \tilde{v} + H_j(t) - H_i(t)$. Based on the induction assumption, there exists a state $(i,\tilde{t},\bar{\tilde{u}},\bar{\tilde{v}}) \in \mathcal{D}_i^\epsilon$ with $\bar{\tilde{u}} \le \tilde{u} + (i-1)\theta$ and $\bar{\tilde{v}} \le \tilde{v}$. According to the execution (before [Elimination]) of $\mathbf{DP_{gen}(Q_B)}$, the state (j,t',u',v') is added to $\mathbf{DP_{gen}(Q_B)}$, where $t' = \max\{\tilde{t}, r_j\} + p$, $u' = \bar{\tilde{u}} + G_j(t') - G_i(t')$, and $v' = \bar{\tilde{v}} + H_j(t') - H_i(t')$. Then after executing [Elimination] of $\mathbf{DP_{gen}(Q_B)}$, there exists a state $(j,t',\bar{u},\bar{v})$ with $\bar{u} \le u' + \theta$ and

$\bar{v} \leq v'$. Based on the facts that $t' = \max\{\tilde{t}, r_j\} + p = t$, $\bar{u} \leq u' + \theta = \bar{\tilde{u}} + G_j(t') - G_i(t') + \theta \leq \tilde{u} + (i-1)\theta + G_j(t') - G_i(t') + \theta = u + i\theta \leq u + (j-1)\theta$, and $\bar{v} \leq v' = \bar{\tilde{v}} + H_j(t') - H_i(t') \leq \tilde{v} + H_j(t') - H_i(t') = v$, the lemma holds for $k = j$ as well. □

Theorem 6.2 *Let the state $(n, t, u^*, v^*) \in \mathcal{D}_n$ characterize an optimal schedule for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$. Algorithm $\mathbf{DP_{gen}(Q_B)}$ finds, in $O(\frac{n^5 F_{\mathcal{UB}}}{\epsilon F_{\mathcal{LB}}})$ time, an approximate schedule defined by the state $(n, t, \bar{u}, \bar{v}) \in \mathcal{D}^\epsilon_n$ with $\bar{u} \leq (1+\epsilon)u^* = (1+\epsilon)F^*$ and $\bar{v} \leq Q_B$.*

Proof. By Theorem 6.1, $\mathbf{DP_2(Q_B)}$ finds an optimal schedule ρ^*, which corresponds to the state $(n, t, u^*, v^*) \in \mathcal{D}_n$ for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$. From the definition of ρ^*, we have $F^* = u^* \leq F_{\mathcal{UB}}$ and $v^* \leq Q_B$. If $(n, t, u^*, v^*) \notin \mathcal{D}^\epsilon_n$, by Lemma 6.1, there exists a state $(n, t, \overline{u^*}, \overline{v^*}) \in \mathcal{D}^\epsilon_n$ with $\overline{u^*} \leq u^* + (n-1)\theta = F^* + \epsilon F_{\mathcal{LB}} \leq (1+\epsilon)F^*$ and $\overline{v^*} \leq v^* \leq Q_B$. Then, after executing Step 5, a state $(n, t, \bar{u}, \bar{v}) \in \mathcal{D}^\epsilon_n$ with $\bar{u} \leq \overline{u^*} \leq (1+\epsilon)F^*$ and $\bar{v} \leq Q_B$ is found.

Next, we establish the time complexity of $\mathbf{DP_{gen}(Q_B)}$. Step 1 takes $O((n-1)(\frac{F_{\mathcal{UB}}}{\epsilon F_{\mathcal{LB}}}+1)) = O(\frac{nF_{\mathcal{UB}}}{\epsilon F_{\mathcal{LB}}})$ time for [Partition], Step 2 takes $O(n^3)$ time for [Pre-processing], and Step 3 takes $O(n)$ time for [Initialization]. Due to [Elimination] in Step 4, at most $O(\frac{n^3 F_{\mathcal{UB}}}{\epsilon F_{\mathcal{LB}}})$ different states are retained in $\mathcal{D}^\epsilon_j$. Because it needs to generate $\mathcal{D}^\epsilon_j$ for $j = 1, 2, \ldots, n$, Step 4 takes $O(\frac{n^5 F_{\mathcal{UB}}}{\epsilon F_{\mathcal{LB}}})$ time. Step 5 takes $O(\frac{n^3 F_{\mathcal{UB}}}{\epsilon F_{\mathcal{LB}}})$ time for [Result]. Therefore, the time complexity of $\mathbf{DP_{gen}(Q_B)}$ is bounded by $O(\frac{n^5 F_{\mathcal{UB}}}{\epsilon F_{\mathcal{LB}}})$. □

Building on the space complexity analysis of $\mathbf{DP_1}$ and integrating the partition procedure, it follows that $\mathbf{DP_{gen}(Q_B)}$ requires $O(\frac{n^4 F_{\mathcal{UB}}}{\epsilon F_{\mathcal{LB}}})$ space. Both the time and space complexity of $\mathbf{DP_{gen}(Q_B)}$ depend on the choice of $F_{\mathcal{LB}}$ and $F_{\mathcal{UB}}$. To this end, we deal with its corresponding auxiliary problem $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$. Let ϕ^* denote an optimal schedule for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$, and let G^* denote its associated objective value, i.e., $G^* = f^A_{\max}(\phi^*) = \max\{f^A_j(\phi^*) : 1 \leq j \leq n_A\}$. By Theorem 3.2, ϕ^* can be determined in $O(n^3 \log n)$ time. If $G^* = 0$, then ϕ^* is also an optimal schedule for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$. In this case, we have $F^* = 0$. Assume that $G^* > 0$, then $F^* > 0$. From the definitions of ρ^* and ϕ^*, we have

$$G^* = f^A_{\max}(\phi^*) \leq f^A_{\max}(\rho^*) \leq f^A_{\max}(\rho^*) = F^* \leq f^A_{\max}(\phi^*) \leq n_A f^A_{\max}(\phi^*) = n_A G^*. \quad (17)$$

By setting $F_{\mathcal{LB}} = G^*$ and $F_{\mathcal{UB}} = n_A G^*$, the time complexity of $\mathbf{DP_{gen}(Q_B)}$ is $O(\frac{n^5 n_A}{\epsilon})$.

To derive tight lower and upper bounds, we employ the BIP proposed by Chubanov et al. (2006). Consider a minimization problem $\mathcal{P}$ with optimal objective value $Z^* > 0$. The application of BIP hinges on the existence of an approximation algorithm, denoted as $\mathcal{A}(X_1, X_2)$, which must satisfy the following two essential features:

(F-1) Given an instance $\mathcal{I}$ of problem $\mathcal{P}$ and two positive values X_1 and X_2, $\mathcal{A}(X_1, X_2)$ outputs a feasible solution with objective value $Z_{\mathcal{A}(X_1,X_2)} \leq X_1 + X_2$ if $Z^* \leq X_2$.

(F-2) The running time of $\mathcal{A}(X_1, X_2)$ is bounded by a polynomial $P(Len, X_2/X_1)$ of two variables, where Len is the binary encoding length of $\mathcal{I}$.

Lemma 6.2 *(Chubanov et al., 2006) If $\mathcal{A}(X_1, X_2)$ satisfies the features (F-1) and (F-2) and $0 < X \leq Z^* \leq Y$, then BIP delivers a value Z_0 for $\mathcal{P}$ with $Z_0 \leq Z^* \leq 3Z_0$ in $O(P(Len, 2)\log\log(Y/X))$ time.*

By Theorem 6.2, $\mathbf{DP_{gen}(Q_B)}$ finds a state $(n, t, \overline{u}, \overline{v}) \in \mathcal{D}_n^\epsilon$ with $\overline{u} \leq F^* + \epsilon F_{\mathcal{LB}} \leq \epsilon F_{\mathcal{LB}} + F_{\mathcal{UB}}$. Therefore, $\mathbf{DP_{gen}(Q_B)}$ satisfies feature (F-1) with $X_1 = \epsilon F_{\mathcal{LB}}$ and $X_2 = F_{\mathcal{UB}}$. Note that the length of $1|\beta^*|\epsilon(f_{\max}^A, f_{\max}^B)$ is $O(n)$ and the time complexity of $\mathbf{DP_{gen}(Q_B)}$ is $O(\frac{n^5 F_{\mathcal{UB}}}{\epsilon F_{\mathcal{LB}}}) = O(\frac{n^5 X_2}{X_1})$. Hence, $\mathbf{DP_{gen}(Q_B)}$ satisfies feature (F-2) as well. This leads to the following lemma.

Lemma 6.3 *For $1|\beta^*|\epsilon(f_{\max}^A, f_{\max}^B)$, a pair of tight bounds with $F_0 \leq F^* \leq 3F_0$ can be achieved via BIP in $O(n^5 \log\log n_A)$ time.*

In view of Theorem 6.2 and Lemma 6.3, the following algorithm is an FPTAS for $1|\beta^*|\epsilon(f_{\max}^A, f_{\max}^B)$.

Algorithm $\mathbf{DP_\epsilon(Q_B)}$. An FPTAS for $1|\beta^*|\epsilon(f_{\max}^A, f_{\max}^B)$

Input: $\mathcal{J}^X = \{J_1^X, J_2^X, \ldots, J_{n_X}^X\}$, p, Q_B, ϵ, r_j^X, w_j^X, d_j^X, f_j^X for $X = A, B$ and $j = 1, 2, \ldots, n_X$.

Step 1. Determine an optimal schedule ϕ^* for $1|\beta^*|\epsilon(f_{\max}^A, f_{\max}^B)$, and let G^* be its associated objective value. If $G^* = 0$, the algorithm is terminated. Otherwise, go to Step 2.

Step 2. Narrow $[G^*, n_A G^*]$ to $[F_0, 3F_0]$ by applying the BIP of Chubanov et al. (2006).

Step 3. Invoke $\mathbf{DP_{gen}(Q_B)}$ with $F_{\mathcal{LB}} = F_0$ and $F_{\mathcal{UB}} = 3F_0$ to find an approximate schedule ρ.

Output: The schedule ϕ^* (if $G^* = 0$) or ρ (if $G^* > 0$).

Theorem 6.3 *For $1|\beta^*|\epsilon(f_{\max}^A, f_{\max}^B)$, Algorithm $\mathbf{DP_\epsilon(Q_B)}$ provides an FPTAS with time complexity $O(n^5(\log\log n_A + \frac{1}{\epsilon}))$.*

Proof. The correctness of $\mathbf{DP_\epsilon(Q_B)}$ is ensured by Theorem 6.2 and Lemma 6.3. By Theorem 3.2, Step 1 takes $O(n^3 \log n)$ time to solve $1|\beta^*|\epsilon(f_{\max}^A, f_{\max}^B)$. By Lemma 6.3, Step 2 takes $O(n^5 \log\log n_A)$ time to apply the BIP. By Theorem 6.2, Step 3 takes $O(n^5/\epsilon)$ time to invoke $\mathbf{DP_{gen}(Q_B)}$. Thus, the time complexity of $\mathbf{DP_\epsilon(Q_B)}$ is $O(n^5(\log\log n_A + \frac{1}{\epsilon}))$. □

Next, we establish the space complexity of $\mathbf{DP_\epsilon(Q_B)}$. In Step 1, computing an optimal schedule requires $O(n^3 \log n)$ space. In Step 2, the application of BIP invokes $\mathbf{DP_{gen}(Q_B)}$ $O(\log\log n_A)$ times, and each invocation requires $O(n^4)$ space to maintain all intermediate states, so the total storage for this step is $O(n^4 \log\log n_A)$. In Step 3, executing $\mathbf{DP_{gen}(Q_B)}$ necessitates $O(n^4/\epsilon)$ space to retain the required state information. Therefore, the total space complexity of $\mathbf{DP_\epsilon(Q_B)}$ is bounded by $O(n^4(\log\log n_A + \frac{1}{\epsilon}))$.

6.3 A super-dual FPTAS for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$

Our super-dual FPTAS for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$ is similar to the FPTAS developed in Section 6.2, but it does not need to construct valid lower and upper bounds.

Algorithm SDDP$_\epsilon$($\mathbf{Q_B}$). A super-dual FPTAS for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$

Input: $\mathcal{J}^X = \{J^X_1, J^X_2, \ldots, J^X_{n_X}\}$, p, Q_B, ϵ, r^X_j, w^X_j, d^X_j, f^X_j for $X = A, B$ and $j = 1, 2, \ldots, n_X$.

Step 1. (**Partition**) Set $\Psi = (1+\epsilon)Q_B$, $\psi = \frac{\epsilon Q_B}{n-1}$, and $y = \lceil \frac{\Psi}{\psi} \rceil = \lceil (n-1)(1+\frac{1}{\epsilon}) \rceil$. Divide the interval $[0, \Psi]$ into y subintervals $\mathcal{I}_k$ with $\mathcal{I}_k = [(k-1)\psi, k\psi)$ for $k = 1, 2, \ldots, y-1$, and $I_y = [(y-1)\psi, \Psi]$.

Step 2. (**Pre-processing**) Same as that in Step 1 of Algorithm **DP$_1$**.

Step 3. (**Initialization**) Set $\mathcal{L}^\epsilon_0 = \{(0,0,0,0)\}$, $\mathcal{L}^\epsilon_j = \emptyset$, and $\mathcal{L}^\epsilon_{j,i} = \emptyset$ for $j = 1, 2, \ldots, n$ and $i = 0, 1, \ldots, j-1$.

Step 4. (**Generation**) Generate $\mathcal{L}^\epsilon_j$ from $\mathcal{L}^\epsilon_0, \mathcal{L}^\epsilon_1, \ldots, \mathcal{L}^\epsilon_{j-1}$.

For $j = 1$ to n **do**

For $i = 0$ to $j-1$ **do**

For each $(i, t, u, v) \in \mathcal{L}^\epsilon_i$ **do**

Set $t' = \max\{t, r_j\} + p$, $u' = u + G_j(t') - G_i(t')$, and $v' = v + H_j(t') - H_i(t')$.

If $t' \le \Lambda$ and $v' \le \Psi$, **then**

Add (j, t', u', v') to $\mathcal{L}^\epsilon_{j,i}$.

End if

End for

End for

Set $\mathcal{L}^\epsilon_j = \mathcal{L}^\epsilon_{j,0} \cup \mathcal{L}^\epsilon_{j,1} \cup \cdots \cup \mathcal{L}^\epsilon_{j,j-1}$.

(**Elimination**) Among all states $(j, t, u, v) \in \mathcal{L}^\epsilon_j$ sharing the same t value and $v \in \mathcal{I}_k$, retain only the one with the smallest u value.

End for

Step 5. (**Result**) Set $\widehat{F} = \min\{u : (n, t, u, v) \in \mathcal{L}^\epsilon_n\}$.

Output: The approximate objective value $\widehat{F}$ and its associated schedule $\widehat{\rho}$.

Analogous to the proof of Lemma 6.1, we obtain the following lemma.

Lemma 6.4 *For any eliminated state $(j,t,u,v) \in \mathcal{D}_j$ with $v \le Q_B$, Algorithm* $\mathbf{SDDP}_\epsilon(\mathbf{Q_B})$ *finds a state $(j,t,\widehat{u},\widehat{v}) \in \mathcal{L}_j^\epsilon$ with $\widehat{u} \le u$ and $\widehat{v} \le v + (j-1)\psi$.*

Theorem 6.4 *Let the state $(n,t,u^*,v^*) \in \mathcal{D}_n$ characterize an optimal schedule for $1|\beta^*|\epsilon(f_{\max}^A, f_{\max}^B)$. Algorithm* $\mathbf{SDDP}_\epsilon(\mathbf{Q_B})$ *finds, in $O(\frac{n^5}{\epsilon})$ time, an approximate schedule defined by the state $(n,t,\widehat{u},\widehat{v}) \in \mathcal{L}_n^\epsilon$ with $\widehat{u} \le u^* = F^*$ and $\widehat{v} \le (1+\epsilon)Q_B$.*

Proof. By Theorem 6.1, $\mathbf{DP_2}(\mathbf{Q_B})$ finds an optimal schedule ρ^*, which corresponds to the state $(n,t,u^*,v^*) \in \mathcal{D}_n$ for $1|\beta^*|\epsilon(f_{\max}^A, f_{\max}^B)$. Therefore, we have $F^* = u^*$ and $v^* \le Q_B$. If $(n,t,u^*,v^*) \notin \mathcal{L}_n^\epsilon$, by Lemma 6.4, there exists a state $(n,t,\widehat{u^*},\widehat{v^*}) \in \mathcal{L}_n^\epsilon$ with $\widehat{u^*} \le u^* = F^*$ and $\widehat{v^*} \le v^* + (n-1)\varphi \le Q_B + \epsilon Q_B = (1+\epsilon)Q_B$. After executing Step 5, a state $(n,t,\widehat{u},\widehat{v}) \in \mathcal{L}_n^\epsilon$ with $\widehat{u} \le \widehat{u^*} \le u^* = F^*$ and $\widehat{v} \le \Psi = (1+\epsilon)Q_B$ is found.

Note that according to the elimination procedure in Step 4, each $\mathcal{L}_j^\epsilon$ contains at most $O(\frac{n^3}{\epsilon})$ different states. Following the analysis of Theorem 6.2, the time complexity of $\mathbf{SDDP}_\epsilon(\mathbf{Q_B})$ is bounded by $O(\frac{n^5}{\epsilon})$. □

Building on the space complexity analysis of $\mathbf{DP_1}$ and integrating the partition procedure, it follows that $\mathbf{SDDP}_\epsilon(\mathbf{Q_B})$ requires $O(\frac{n^4}{\epsilon})$ space.

Remark 6.1 *Given an instance of $1|\beta^*|\epsilon(f_{\max}^A, f_{\max}^B)$ and a fixed $\epsilon > 0$, there may be no feasible schedule ρ with $f_{\max}^B(\rho) \le Q_B$, while a feasible schedule ρ' satisfying $f_{\max}^B(\rho') \le (1+\epsilon)Q_B$ might still exist.*

7 A $(1, 1+\epsilon)$-approximate POF for $1|\beta^*|\#(f_{\max}^A, f_{\max}^B)$

In this section, we propose an approximate POF for $1|\beta^*|\#(f_{\max}^A, f_{\max}^B)$. To simplify the notation, let $\mathcal{Q}$ denote the set of POF for $1|\beta^*|\#(f_{\max}^A, f_{\max}^B)$.

Definition 7.1 *A set $\mathcal{Q}_{(\lambda_1,\lambda_2)}$ for $1|\beta^*|\#(f_{\max}^A, f_{\max}^B)$ is a (λ_1,λ_2)-approximate POF if, for each point $(Z_A,Z_B) \in \mathcal{Q}$, there exists a point $(Z'_A,Z'_B) \in \mathcal{Q}_{(\lambda_1,\lambda_2)}$ with $Z'_A \le \lambda_1 Z_A$ and $Z'_B \le \lambda_2 Z_B$, and there exists an associated feasible schedule ρ with $f_{\max}^A(\rho) = Z'_A$ and $f_{\max}^B(\rho) = Z'_B$.*

In view of Theorems 6.3 and 6.4, we obtain the following remark.

Remark 7.1 *If (Z_A,Z_B) is a Pareto-optimal point for $1|\beta^*|\#(f_{\max}^A, f_{\max}^B)$, then Algorithm* $\mathbf{DP}_\epsilon(\mathbf{Z_B})$ *finds a schedule π with $f_{\max}^A(\pi) \le (1+\epsilon)Z_A$ and $f_{\max}^B(\pi) \le Z_B$ in $O(n^5(\log\log n_A + \frac{1}{\epsilon}))$ time, and Algorithm* $\mathbf{SDDP}_\epsilon(\mathbf{Z_B})$ *finds a schedule ρ with $f_{\max}^A(\rho) \le Z_A$ and $f_{\max}^B(\rho) \le (1+\epsilon)Z_B$ in $O(\frac{n^5}{\epsilon})$ time.*

Based on Remark 7.1, if a Pareto-optimal point $(Z_A, Z_B) \in \mathcal{Q}$ is selected, then $\mathbf{SDDP}_\epsilon(\mathbf{Z_B})$ will deliver an approximate schedule ρ with $f^A_{\max}(\rho) \le Z_A$ and $f^B_{\max}(\rho) \le (1+\epsilon)Z_B$. Although (Z_A, Z_B) cannot be determined in advance, a good approximate schedule can still be achieved by using a point close to (Z_A, Z_B).

Theorem 7.1 *Let (Z_A, Z_B) be a Pareto-optimal point for $1|\beta^*|\#(f^A_{\max}, f^B_{\max})$, and let $\epsilon' = \sqrt{1+\epsilon} - 1$. If $Z'_B \in [Z_B, (1+\epsilon')Z_B]$, then Algorithm $\mathbf{SDDP}_{\epsilon'}(\mathbf{Z'_B})$ finds an approximate schedule ρ with $f^A_{\max}(\rho) \le Z_A$ and $f^B_{\max}(\rho) \le (1+\epsilon)Z_B$.*

Proof. Given that $(Z_A, Z_B) \in \mathcal{Q}$, Theorem 6.1 ensures that applying $\mathbf{DP_2}(\mathbf{Z_B})$ to $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$ with $Q_B = Z_B$ yields an optimal schedule ρ^*, which corresponds to a state $(n, t, Z_A, Z_B) \in \mathcal{D}_n$. If $Z'_B \in [Z_B, (1+\epsilon')Z_B]$, then (n, t, Z_A, Z_B) remains a feasible state in $\mathcal{D}_n$ for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$ with $Q_B = Z'_B$. By Lemma 6.4, if this state is eliminated during the execution of $\mathbf{SDDP}_{\epsilon'}(\mathbf{Z'_B})$, a new state $(n, t, \widehat{Z_A}, \widehat{Z_B}) \in \mathcal{L}^\epsilon_n$ is identified, where $\widehat{Z_A} \le Z_A$, $\widehat{Z_B} \le Z_B + (n-1)\varphi \le (1+\epsilon')Z'_B$. After executing Step 5, $\mathbf{SDDP}_{\epsilon'}(\mathbf{Z'_B})$ constructs an approximate schedule ρ, defined by a state $(n, \widehat{t}, \widehat{u}, \widehat{v}) \in \mathcal{L}^\epsilon_n$, where $\widehat{u} \le \widehat{Z_A} \le Z_A$ and $\widehat{v} \le (1+\epsilon')Z'_B$. Given that $Z'_B \le (1+\epsilon')Z_B$ and $\epsilon' = \sqrt{1+\epsilon} - 1$, we further have $f^A_{\max}(\rho) = \widehat{u} \le Z_A$ and $f^B_{\max}(\rho) = \widehat{v} \le (1+\epsilon')Z'_B \le (1+\epsilon')^2 Z_B = (1+\epsilon)Z_B$. □

Recall that $\Delta_B = \sum_{j=1}^{n_B} f^B_j(\Lambda)$ is an upper bound on $f^B_{\max}$. Let δ_B denote the minimum objective value of $1|\beta^*|f^B_{\max}$ over the job set $\mathcal{J}^B$. Clearly, δ_B is a lower bound on $f^B_{\max}$ of $1|\beta^*|\#(f^A_{\max}, f^B_{\max})$. Below, we provide a bi-criteria approximation algorithm for $1|\beta^*|\#(f^A_{\max}, f^B_{\max})$, which constructs a $(1, 1+\epsilon)$-approximate POF.

Algorithm $\mathbf{APF}_\epsilon$. A $(1, 1+\epsilon)$-approximate POF for $1|\beta^*|\#(f^A_{\max}, f^B_{\max})$

Input: $\mathcal{J}^X = \{J^X_1, J^X_2, \ldots, J^X_{n_X}\}$, p, Δ_B, δ_B, ϵ, $\epsilon' = \sqrt{1+\epsilon} - 1$, r^X_j, w^X_j, d^X_j, f^X_j for $X = A, B$ and $j = 1, 2, \ldots, n_X$.

Step 1. If $\delta_B > 0$, set $\mathcal{X} = \{(1+\epsilon')^k \delta_B : k = 0, 1, \ldots, \mu\}$, otherwise set $\mathcal{X} = \{0\} \cup \{(1+\epsilon')^k : k = 0, 1, \ldots, \nu\}$, where μ is the smallest integer with $(1+\epsilon')^\mu \delta_B \ge \Delta_B$, and ν is the smallest integer with $(1+\epsilon')^\nu \ge \Delta_B$. Set $\mathcal{Q}_{(1,1+\epsilon)} = \emptyset$.

Step 2. For each $Z_B \in \mathcal{X}$, invoke $\mathbf{SDDP}_{\epsilon'}(\mathbf{Z_B})$ to find a schedule ρ_{Z_B} for $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$ with $Q_B = Z_B$ (if possible), and add $(f^A_{\max}(\rho_{Z_B}), f^B_{\max}(\rho_{Z_B}))$ to $\mathcal{Q}_{(1,1+\epsilon)}$.

Step 3. For any two states (u, v) and (u', v') in $\mathcal{Q}_{(1,1+\epsilon)}$ such that $u \le u'$ and $v \le v'$, retain only the first one.

Output: The $(1, 1+\epsilon)$-approximate POF $\mathcal{Q}_{(1,1+\epsilon)}$.

In $\mathbf{APF}_\epsilon$, if $\delta_B > 0$, then $\mu = \lceil \log_{1+\epsilon'} \frac{\Delta_B}{\delta_B} \rceil = O(\frac{1}{\epsilon} \log_2 \frac{\Delta_B}{\delta_B})$; otherwise $\nu = \lceil \log_{1+\epsilon'} \Delta_B \rceil = O(\frac{\log_2 \Delta_B}{\epsilon})$. Since each execution of $\mathbf{SDDP}_{\epsilon'}(\mathbf{Z_B})$ requires $O(\frac{n^5}{\epsilon})$ time, by applying Theorem 7.1, we obtain the following theorem.

Theorem 7.2 *Algorithm* $\mathbf{APF}_\epsilon$ *finds a* $(1, 1+\epsilon)$*-approximate POF for* $1|\beta^*|\#(f^A_{\max}, f^B_{\max})$ *in* $O(\frac{n^5}{\epsilon^2}\log_2 \frac{\Delta_B}{\max\{\delta_B,1\}})$ *time.*

In $\mathbf{APF}_\epsilon$, each execution of $\mathbf{SDDP}_{\epsilon'}(\mathbf{Z_B})$ in Step 2 requires $O(n^4/\epsilon)$ space to store all essential states. Since all possible values of Z_B must be enumerated, the total space complexity of $\mathbf{APF}_\epsilon$ is $O(\frac{n^4}{\epsilon^2}\log_2 \frac{\Delta_B}{\max\{\delta_B,1\}})$.

Remark 7.2 *In Step 2 of* $\mathbf{APF}_\epsilon$*, if* $\mathbf{DP}_\epsilon(\mathbf{Z_B})$ *is invoked to solve* $1|\beta^*|\epsilon(f^A_{\max}, f^B_{\max})$*, then a* $(1+\epsilon, 1+\epsilon)$*-approximate POF is generated for* $1|\beta^*|\#(f^A_{\max}, f^B_{\max})$ *in* $O(\frac{n^5(\log\log n_A + 1/\epsilon)}{\epsilon^2}\log_2 \frac{\Delta_B}{\max\{\delta_B,1\}})$ *time.*

8 Numerical study

We conduct a series of computational experiments to evaluate the performance (running times) of the proposed algorithms for $1|\beta^*|\#(f^A_{\max}, f^B_{\max})$, as discussed in Sections 5 and 7. All programs were implemented in Matlab and run on a PC with a 12th Gen Intel(R) Core(TM) i7-1260P processor (2.10 GHz) and 16 GB of RAM.

Note that $\mathbf{DP_1}$ is used to find the exact POF (i.e., the set of all Pareto optimal points) for $1|\beta^*|\#(f^A_{\max}, f^B_{\max})$, while $\mathbf{APF}_\epsilon$ is employed to find a $(1, 1+\epsilon)$-approximate POF. The complexity of these algorithms varies across different combinations of scheduling criteria. In our analysis, we focus on three representative problems:

- Problem $\mathbf{P_1}$: $1|\beta^*|\#(\sum w^A_j C^A_j, \sum w^B_j C^B_j)$;
- Problem $\mathbf{P_2}$: $1|\beta^*|\#(\sum w^A_j C^A_j, \sum w^B_j Y^B_j)$;
- Problem $\mathbf{P_3}$: $1|\beta^*|\#(\sum w^A_j T^A_j, \sum w^B_j U^B_j)$.

These problems involve classical scheduling criteria: $\sum w_j C_j$, $\sum w_j Y_j$, $\sum w_j T_j$, and $\sum w_j U_j$. To evaluate the performance of the algorithms, we generated random instances of varying sizes by considering a range of values for key parameters. Let $r_{\max}$ denote the maximum release date and $w_{\max}$ denote the maximum weight of jobs in $\mathcal{J}$.

Following the parameter settings in Hall and Posner (2001), we employed the assumptions detailed below to create diverse sets of parameter values. The number of jobs n was chosen from the set $\{20, 50, 100, 150, 200\}$; the processing time p was selected from $\{5, 15\}$; the release date r^X_j was uniformly drawn from the integer range $[1, r_{\max}]$, where $r_{\max} \in \{50, 100\}$; the weight w^X_j was uniformly drawn from the integer range $[1, w_{\max}]$, where $w_{\max} \in \{20, 50\}$; and the due date d^X_j was uniformly drawn from the integer range $[r^X_j, \Lambda]$, where $\Lambda = r_{\max} + 2p - 1$. The rationale for selecting due date is that, in an optimal solution, all jobs can be completed by time Λ. For each combination of parameters n (with $n_A = n_B = n/2$), p, $w_{\max}$, and $r_{\max}$, we simulated and solved 20 problem instances. Tables 1, 2, and 3 report the average and maximum running times (in seconds) for $\mathbf{DP_1}$ and $\mathbf{APF}_\epsilon$, where $\epsilon \in \{0.1, 0.4\}$

is the relative error. Instances that could not be solved due to exceeding the 16 GB memory limit are indicated with "–" in the tables.

Table 1: Average and maximum running times (sec) of $\mathbf{DP_1}$ and $\mathbf{APF}_\epsilon$ for $\mathbf{P_1}$

			$r_{\max} = 50$						$r_{\max} = 100$					
			$\mathbf{DP_1}$		$\mathbf{AFP_{0.1}}$		$\mathbf{AFP_{0.4}}$		$\mathbf{DP_1}$		$\mathbf{AFP_{0.1}}$		$\mathbf{AFP_{0.4}}$	
n	p	$w_{\max}$	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
20	5	20	1.79	4.36	0.06	0.11	0.02	0.04	18.60	46.41	0.16	0.25	0.05	0.09
20	5	50	4.21	8.71	0.07	0.14	0.02	0.04	27.81	55.32	0.19	0.29	0.06	0.11
20	15	20	1.66	2.61	0.03	0.04	0.01	0.02	5.84	10.34	0.07	0.10	0.03	0.04
20	15	50	4.25	8.11	0.04	0.09	0.01	0.03	13.54	26.87	0.08	0.15	0.03	0.05
50	5	20	31.07	42.10	0.61	1.25	0.30	0.51	150.61	217.89	2.86	4.48	0.81	2.36
50	5	50	76.72	105.74	0.86	1.85	0.34	0.69	376.22	601.31	2.16	3.51	1.08	2.59
50	15	20	38.81	58.42	0.50	0.67	0.16	0.23	135.77	265.23	1.56	2.87	0.55	1.13
50	15	50	90.42	126.71	0.54	0.88	0.17	0.27	389.34	768.95	2.13	4.58	0.79	2.79
100	5	20	264.33	334.42	24.19	31.86	8.61	12.09	1138.96	1507.84	62.57	81.36	19.97	25.72
100	5	50	625.09	719.82	28.82	48.75	9.87	31.96	4335.28	6931.52	68.46	92.37	21.07	27.67
100	15	20	331.71	385.79	8.17	10.30	3.37	4.43	1931.84	3618.74	36.07	45.75	14.20	16.63
100	15	50	954.98	1106.62	8.93	11.50	3.47	5.23	3635.43	5685.09	40.34	50.70	14.11	17.26
150	5	20	396.10	464.36	79.22	98.43	29.48	36.67	4573.46	5954.6	259.53	319.12	82.52	103.19
150	5	50	2413.34	2922.72	100.17	137.3	32.41	47.55	–	–	290.00	338.91	89.45	125.58
150	15	20	795.67	1271.26	26.61	31.28	11.34	13.48	5026.51	6619.89	128.49	167.75	51.33	65.05
150	15	50	3646.48	6662.78	31.02	48.08	154.01	25.15	–	–	163.72	200.88	59.55	86.84
200	5	20	983.04	1414.55	206.89	256.9	78.42	98.12	–	–	650.64	773.28	207.40	251.81
200	5	50	6462.08	8922.55	277.95	395.22	86.43	121.34	–	–	711.46	871.91	231.58	313.43
200	15	20	1842.41	2450.58	107.74	268.42	41.21	140.75	–	–	332.6	366.51	132.61	210.69
200	15	50	–	–	135.97	341.4	40.22	135.83	–	–	385.19	486.42	181.83	297.81

Focusing on problem $\mathbf{P_1}$ (Table 1), we observe that $\mathbf{DP_1}$ exhibits a sharp increase in computational demand as instance size grows. For instances with $n = 20$, the maximum running time does not exceed 56 seconds. However, for instances with $n = 100$ under the worst-case scenario ($p = 5$, $w_{\max} = 50$, and $r_{\max} = 100$), the running time escalates dramatically to 6931.52 seconds. For instances with $n = 200$, $\mathbf{DP_1}$ becomes entirely impractical. In contrast, $\mathbf{APF}_\epsilon$ achieves significantly shorter average and maximum running times under identical conditions. Notably, increasing the value of ϵ from 0.1 to 0.4 further reduces the running time, which aligns with the expected decrease in the number of iterations required. Specifically, for all 200-job instances, $\mathbf{APF_{0.1}}$ can identify a $(1, 1.1)$-approximate POF in no more than 872 seconds, while $\mathbf{APF_{0.4}}$ can find a $(1, 1.4)$-approximate POF in no more than 314 seconds.

Table 2: Average and maximum running times (sec) of $\mathbf{DP_1}$ and $\mathbf{APF}_\epsilon$ for $\mathbf{P_2}$

			$r_{\max} = 50$						$r_{\max} = 100$					
			$\mathbf{DP_1}$		$\mathbf{AFP_{0.1}}$		$\mathbf{AFP_{0.4}}$		$\mathbf{DP_1}$		$\mathbf{AFP_{0.1}}$		$\mathbf{AFP_{0.4}}$	
n	p	$w_{\max}$	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
20	5	20	0.11	0.28	0.03	0.06	0.01	0.02	0.21	0.42	0.03	0.06	0.02	0.02
20	5	50	0.21	0.48	0.03	0.08	0.01	0.02	0.76	1.29	0.04	0.07	0.02	0.02
20	15	20	0.19	0.53	0.02	0.04	0.01	0.01	0.77	2.43	0.03	0.04	0.01	0.02
20	15	50	0.47	0.93	0.03	0.04	0.01	0.01	1.15	2.65	0.05	0.19	0.02	0.04
50	5	20	2.25	3.67	0.62	1.56	0.32	0.79	4.66	9.12	1.16	1.94	0.56	1.21
50	5	50	5.58	17.81	0.72	1.98	0.42	1.14	12.71	22.99	1.66	3.32	0.65	1.44
50	15	20	5.07	10.28	0.26	0.57	0.11	0.24	12.11	28.63	1.14	1.86	0.49	0.91
50	15	50	11.87	17.62	0.31	0.73	0.13	0.35	33.31	59.93	1.44	2.26	0.59	1.21
100	5	20	16.25	22.32	7.87	9.79	4.54	5.77	39.55	53.43	21.73	27.28	13.36	17.77
100	5	50	38.22	56.33	12.04	19.03	5.21	9.12	100.48	127.43	36.47	53.75	16.47	28.45
100	15	20	43.98	61.87	3.74	4.49	1.48	2.07	114.97	186.92	18.86	25.79	9.15	13.82
100	15	50	94.48	155.95	4.93	7.58	1.94	2.96	267.04	359.45	22.16	29.59	10.06	15.97
150	5	20	56.42	79.86	35.69	51.18	19.73	32.54	147.72	184.99	108.77	135.08	69.15	76.31
150	5	50	151.59	239.59	57.68	83.57	26.43	39.55	350.09	456.39	162.22	199.44	76.21	92.89
150	15	20	178.93	254.54	13.37	15.89	6.12	8.25	421.71	564.95	99.37	120.02	42.25	59.62
150	15	50	457.14	625.57	26.58	36.65	10.14	14.59	938.12	1284.6	115.98	146.68	47.67	64.95
200	5	20	135.03	182.55	85.14	133.16	52.97	78.65	337.43	400.46	256.79	310.62	166.47	186.84
200	5	50	350.73	514.98	144.57	228.11	65.47	88.69	739.18	1008.12	378.02	407.26	178.42	211.12
200	15	20	376.25	542.2	38.93	58.43	16.88	27.15	1193.22	1578.12	317.91	419.13	138.15	185.04
200	15	50	886.04	1285.9	43.36	65.66	21.16	38.88	2370.71	3279.98	342.55	524.27	174.62	249.42

Regarding problem $\mathbf{P_2}$, the analysis of Table 2 reveals that $\mathbf{DP_1}$ effectively solves all instances with $n = 50$ in no more than 60 seconds, while requiring no more than 360 seconds for instances with $n = 100$. The maximum running time observed does not exceed 3280 seconds for instances with $n = 200$. In contrast, for all 200-job instances, $\mathbf{APF_{0.1}}$ successfully identifies a $(1, 1.1)$-approximate POF within 525 seconds, whereas $\mathbf{APF_{0.4}}$ achieves a $(1, 1.4)$-approximate POF within 250 seconds.

Turning to problem $\mathbf{P_3}$ (Table 3), both $\mathbf{DP_1}$ and $\mathbf{APF}_\epsilon$ demonstrate high efficiency across all generated instances. Remarkably, even for instances with $n = 200$, $\mathbf{DP_1}$ requires no more than 354 seconds, and $\mathbf{APF_{0.1}}$ can identify a $(1, 1.1)$-approximate POF within 148 seconds.

Based on the results from Tables 1, 2, and 3, we observe that: (i) the running times for both $\mathbf{DP_1}$ and $\mathbf{APF}_\epsilon$ are sensitive to the parameters n, $r_{\max}$, and $w_{\max}$ as they increase; (ii) $\mathbf{DP_1}$ exhibits significantly longer running times compared to $\mathbf{APF}_\epsilon$ (e.g., $\epsilon = 0.4$), particularly for larger instance sizes and higher values of $r_{\max}$ and $w_{\max}$. Moreover, the computational effort varies considerably across problems $\mathbf{P_1}$, $\mathbf{P_2}$ and $\mathbf{P_3}$, even with identical parameter configurations. For example, with the configuration of $n = 100$, $p = 5$, $w_{\max} = 20$,

and $r_{\max} = 100$ across their respective 20 generated instances, $\mathbf{DP_1}$ requires approximately 1138.96 seconds for $\mathbf{P_1}$, 39.55 seconds for $\mathbf{P_2}$, and 10.01 seconds for $\mathbf{P_3}$. In contrast, $\mathbf{APF_{0.1}}$ and $\mathbf{APF_{0.4}}$ exhibit significantly lower computational demands, with average times of 62.57 and 19.97 seconds for $\mathbf{P_1}$, 21.73 and 13.36 seconds for $\mathbf{P_2}$, and 7.14 and 5.38 seconds for $\mathbf{P_3}$, respectively. Additionally, the specific structures related to the scheduling criteria significantly influence computational effort. For instances with $150 \leq n \leq 200$, $\mathbf{DP_1}$ is the preferred algorithm when one agent wants to minimize its weighted number of tardy jobs, while $\mathbf{APF}_\epsilon$ is favored when both agents aim to minimize their weighted completion time.

Table 3: Average and maximum running times (sec) of $\mathbf{DP_1}$ and $\mathbf{APF}_\epsilon$ for $\mathbf{P_3}$

			$r_{\max} = 50$						$r_{\max} = 100$					
			$\mathbf{DP_1}$		$\mathbf{AFP_{0.1}}$		$\mathbf{AFP_{0.4}}$		$\mathbf{DP_1}$		$\mathbf{AFP_{0.1}}$		$\mathbf{AFP_{0.4}}$	
n	p	$w_{\max}$	avg	max	avg	max	avg	max	avg	max	avg	max	avg	max
20	5	20	0.03	0.07	0.01	0.03	0.01	0.01	0.07	0.23	0.03	0.05	0.01	0.02
20	5	50	0.06	0.12	0.02	0.02	0.01	0.01	0.11	0.25	0.03	0.04	0.01	0.02
20	15	20	0.03	0.08	0.01	0.04	0.01	0.01	0.04	0.11	0.02	0.03	0.01	0.01
20	15	50	0.04	0.09	0.01	0.02	0.01	0.01	0.07	0.17	0.02	0.03	0.01	0.01
50	5	20	0.42	0.69	0.19	0.29	0.11	0.25	1.03	1.62	0.55	0.85	0.27	0.41
50	5	50	1.22	2.12	0.38	0.52	0.18	0.28	2.63	4.36	0.71	1.22	0.33	0.65
50	15	20	0.45	0.82	0.15	0.26	0.06	0.14	0.85	1.36	0.39	0.71	0.17	0.39
50	15	50	1.01	1.68	0.29	0.48	0.10	0.13	2.72	5.31	0.57	0.82	0.23	0.41
100	5	20	3.86	5.72	2.77	3.77	1.61	2.38	10.01	15.12	7.14	9.95	5.38	6.91
100	5	50	8.56	12.3	3.82	5.63	2.21	3.51	26.3	40.67	10.97	14.86	7.33	10.70
100	15	20	3.85	6.44	1.72	2.16	0.56	0.76	9.14	12.36	5.45	6.81	2.97	4.42
100	15	50	10.42	13.92	2.34	3.06	0.71	1.04	24.03	32.55	8.23	10.18	4.81	6.42
150	5	20	12.14	16.82	9.87	12.84	6.88	9.55	31.49	42.06	26.56	31.18	19.98	25.97
150	5	50	33.94	47.43	16.65	23.81	11.6	17.11	74.85	96.83	43.14	54.75	31.48	43.32
150	15	20	14.62	31.98	6.06	7.44	2.40	3.44	29.69	39.37	24.83	29.09	15.46	20.96
150	15	50	29.52	41.12	6.69	8.13	2.58	3.92	69.96	96.61	33.31	45.62	19.57	31.35
200	5	20	30.37	38.22	24.53	31.11	18.14	24.51	76.64	113.06	60.89	86.73	54.63	72.69
200	5	50	75.81	93.01	40.43	52.87	30.09	42.42	163.82	216.09	109.38	147.59	84.46	103.37
200	15	20	31.94	48.13	13.69	17.85	6.62	10.81	70.16	94.37	53.97	81.56	35.51	48.99
200	15	50	76.28	109.04	16.24	20.18	7.41	11.72	176.81	353.83	79.62	114.47	55.19	83.93

9 Managerial insights

In this study, we explore a two-agent parallel-batching scheduling model with dynamic job arrivals and equal processing times, applicable to industries such as semiconductor and automotive manufacturing. Our findings, based on theoretical analyses and numerical experiments, provide several key managerial insights for guiding actual production.

- **Sensitivity to critical parameter.** The computational effort is highly sensitive to

several parameters, including the number of jobs n, the maximum release date $r_{\max}$, and the maximum job weight $w_{\max}$. High values of $r_{\max}$ can lead to increased idle times, more processing batches, and larger objective values. Similarly, large job weights complicate scheduling and increase objective values. Thus, reducing job weights proportionally and clustering jobs with similar release dates can facilitate the construction of approximate schedules.

• **Balancing competing objectives of the two agents.** The POF provides a systematic framework to evaluate trade-offs between conflicting objectives of the two agents. When one agent's objective is of the max-form, all problem variants are computationally tractable and can be optimally solved in polynomial time. In contrast, when both agent's objectives are of the sum-form, most variants are $\mathcal{NP}$-hard. We designed a pseudo-polynomial-time algorithm ($\mathbf{DP_1}$) and a $(1, 1+\epsilon)$-approximate POF ($\mathbf{APF}_\epsilon$) for the Pareto problem $1|\beta^*|\#(f^A_{\text{sum}}, f^B_{\text{sum}})$. Our computational experiments indicate that these algorithms can be effectively applied in production settings for instances with $n \leq 200$. For the linear-combination models, production managers can apply the DP algorithm from Cheng et al. (2005) to solve $1|\beta^*|\lambda f^A_{\text{sum}} + (1-\lambda) f^B_{\text{sum}}$, and the approach from Section 3.4 for $1|\beta^*|\lambda\gamma^A + (1-\lambda) f^B_{\max}$, where $\gamma^A \in \{f_{\max}, f_{\text{sum}}\}$. For the restricted problem $1|\beta^*|\epsilon(f^A_{\text{sum}}, f^B_{\text{sum}})$, employing the exact $\mathbf{DP_2(Q_B)}$ for small-scale instances (e.g., $n \leq 50$) is recommended, $\mathbf{SDDP}_\epsilon\mathbf{(Q_B)}$ is preferred if a small error can be tolerated for the bound Q_B, while $\mathbf{DP}_\epsilon\mathbf{(Q_B)}$ can be used if $f^B_{\text{sum}} \leq Q_B$ must be satisfied.

• **Algorithm selection strategy.** The computational efficiency of algorithms varies significantly based on specific structures related to the scheduling criteria, instance sizes and precision requirements. For instances with $n \leq 50$, the exact algorithms $\mathbf{DP_1}$ and $\mathbf{DP_2(Q_B)}$ are suitable for achieving precise decision-making for $1|\beta^*|\#(f^A_{\text{sum}}, f^B_{\text{sum}})$ and $1|\beta^*|\epsilon(f^A_{\text{sum}}, f^B_{\text{sum}})$, respectively. However, for instances with $150 \leq n \leq 200$, exact algorithms become impractical due to the exponential growth in runtime and memory demands. In these cases, based on the specific combination of objective functions and problem variants, the approximation algorithms $\mathbf{APF}_\epsilon$, $\mathbf{DP}_\epsilon\mathbf{(Q_B)}$, and $\mathbf{SDDP}_\epsilon\mathbf{(Q_B)}$ provide a practical balance between solution quality and computational efficiency. For instance, with respect to $1|\beta^*|\#(\sum w^A_j C^A_j, \sum w^B_j C^B_j)$, the scenario where $n = 200$, $p = 5$, $w_{\max} = 50$, and $r_{\max} = 100$ allows $\mathbf{APF_{0.1}}$ to achieve a $(1, 1.1)$-approximate POF within 872 seconds, while $\mathbf{APF_{0.4}}$ yields a $(1, 1.4)$-approximate POF within 314 seconds. On the other hand, Table 3 implies that $\mathbf{DP_1}$ can solve all 200-job instances optimally within 6 minutes for $1|\beta^*|\#(\sum w^A_j T^A_j, \sum w^B_j U^B_j)$. Therefore, production managers should select algorithms based on the problem's specific characteristics and operational requirements: prioritizing exact algorithms for small-scale instances, employing approximation algorithms for larger instances, and adjusting the precision parameter ϵ to balance solution accuracy against computational efficiency.

10 Conclusions

We addressed the scheduling problem with two competing agents, release dates, and equal processing times on an unbounded PBPM. For agent X ($X \in \{A, B\}$), the scheduling criterion was either the max-form $f_{\max}^X = \max_{1\le j\le n_X}\{f_j^X(C_j^X)\}$ or the sum-form $f_{\text{sum}}^X = \sum_{j=1}^{n_X} f_j^X(C_j^X)$. We characterized the structural properties inherent to the optimal solutions of the general problem and summarized the main complexity results in Table 4. Additionally, we designed an $O(n^5(\log\log n_A + \frac{1}{\epsilon}))$-time FPTAS and an $O(\frac{n^5}{\epsilon})$-time super-dual FPTAS for $1|\beta^*|\epsilon(f_{\text{sum}}^A, f_{\text{sum}}^B)$. Furthermore, we proposed a $(1, 1+\epsilon)$-approximate POF for $1|\beta^*|\#(f_{\text{sum}}^A, f_{\text{sum}}^B)$. To assess the performance of our developed algorithms, we conducted a comprehensive set of numerical experiments.

Table 4: Complexity results for $1|\beta^*|\{\gamma^A, \gamma^B\}$

Problem	Complexity	Reference
$1\|\beta^*\|\lambda f_{\text{sum}}^A + (1-\lambda) f_{\text{sum}}^B$	$O(n^3)$	Theorem 2.1
$1\|\beta^*, p=1\|\{\gamma^A, \gamma^B\}$	$O(n\log n)$	Theorem 2.2
$1\|\beta^*\|\epsilon(f_{\max}^A, f_{\max}^B)$	$O(n^3)$	Theorem 3.1
$1\|\beta^*\|\epsilon(f_{\text{sum}}^A, f_{\max}^B)$	$O(n^3)$	Theorem 3.1
$1\|\beta^*\|\epsilon(f_{\max}^A, f_{\text{sum}}^B)$	$O(n^3\log n)$	Theorem 3.2
$1\|\beta^*, f_{\max}^A \le Q_A, f_{\max}^B \le Q_B\|-$	$O(n^2)$	Theorem 3.3
$1\|\beta^*\|\#(f_{\max}^A, f_{\max}^B)$	$O(n^5 n_B)$	Theorem 3.4
$1\|\beta^*\|\#(f_{\text{sum}}^A, f_{\max}^B)$	$O(n^5 n_B)$	Theorem 3.4
$1\|\beta^*\|\lambda f_{\max}^A + (1-\lambda) f_{\max}^B$	$O(n^5 n_B)$	Corollary 3.1
$1\|\beta^*\|\lambda f_{\text{sum}}^A + (1-\lambda) f_{\max}^B$	$O(n^5 n_B)$	Corollary 3.2
$1\|\beta^*, \sum w_j^A C_j^A \le Q_A, \sum w_j^B C_j^B \le Q_B\|-$	$\mathcal{NP}$-complete	Theorem 4.1
$1\|\beta^*, \sum w_j^A U_j^A \le Q_A, \sum w_j^B U_j^B \le Q_B\|-$	$\mathcal{NP}$-complete	Theorem 4.2
$1\|\beta^*, \sum Y_j^A \le Q_A, \sum Y_j^B \le Q_B\|-$	$\mathcal{NP}$-complete	Theorem 4.3
$1\|\beta^*, \sum T_j^A \le Q_A, \sum Y_j^B \le Q_B\|-$	$\mathcal{NP}$-complete	Corollary 4.1
$1\|\beta^*, \sum T_j^A \le Q_A, \sum T_j^B \le Q_B\|-$	$\mathcal{NP}$-complete	Corollary 4.2
$1\|\beta^*, \sum C_j^A \le Q_A, \sum w_j^B U_j^B \le Q_B\|-$	$\mathcal{NP}$-complete	Theorem 4.4
$1\|\beta^*, \sum Y_j^A \le Q_A, \sum w_j^B U_j^B \le Q_B\|-$	$\mathcal{NP}$-complete	Theorem 4.5
$1\|\beta^*, \sum T_j^A \le Q_A, \sum w_j^B U_j^B \le Q_B\|-$	$\mathcal{NP}$-complete	Corollary 4.3
$1\|\beta^*, \sum C_j^A \le Q_A, \sum Y_j^B \le Q_B\|-$	$\mathcal{NP}$-complete	Theorem 4.6
$1\|\beta^*, \sum C_j^A \le Q_A, \sum T_j^B \le Q_B\|-$	$\mathcal{NP}$-complete	Corollary 4.4
$1\|\beta^*, \sum C_j^A \le Q_A, \sum C_j^B \le Q_B\|-$	open	
$1\|\beta^*, \sum C_j^A \le Q_A, \sum w_j^B C_j^B \le Q_B\|-$	open	
$1\|\beta^*\|\#(f_{\text{sum}}^A, f_{\text{sum}}^B)$	$O(n^4\Delta)$	Theorem 5.1
$1\|\beta^*\|\#(f_{\text{sum}}^A, \sum U_j^B)$	$O(n^4 n_B)$	Corollary 5.2
$1\|\beta^*\|\epsilon(f_{\text{sum}}^A, f_{\text{sum}}^B)$	$O(n^4 Q_B)$	Theorem 6.1

$\Delta = \min\{\sum_{j=1}^{n_A} f_j^A(\Lambda), \sum_{j=1}^{n_B} f_j^A(\Lambda)\}$, where $\Lambda = r_{\max}(\mathcal{J}) + 2p - 1$.

Future research could extend this work in several directions. First, determining the computational complexity of $1|\beta^*, \sum C_j^A \le Q_A, \sum C_j^B \le Q_B|-$ and $1|\beta^*, \sum C_j^A \le$

$Q_A, \sum w_j^B C_j^B \leq Q_B|-$ remains an open challenge, specifically regarding their classification as polynomially solvable or $\mathcal{NP}$-complete. Second, numerical tests demonstrate that $\mathbf{DP_1}$ and $\mathbf{APF}_\epsilon$ can solve instances with $n \leq 150$ within a reasonable time, memory limitations hinder their applicability to large-scale instances (e.g., $n \geq 300$). Therefore, developing exact or heuristic algorithms with reduced time and space complexity for such cases is critical. Third, extending the framework to $h \geq 3$ competing agents presents both theoretical and practical opportunities. Key observations include: (i) if the scheduling criterion of agent k is $f_{\max}^k$ for $k = 2, 3, \ldots, h$, then the h-agent problem $1|\beta^*|\epsilon(\gamma^A, f_{\max}^2, f_{\max}^3, \ldots, f_{\max}^h)$ reduces to the two-agent case $1|\beta^*|\epsilon(\gamma^A, f_{\max}^B)$; (ii) when h is fixed, the exact DP algorithm for $1|\beta^*|\#(f_{\text{sum}}^A, f_{\text{sum}}^B)$ can be extended to solve $1|\beta^*|\#(f_{\text{sum}}^1, f_{\text{sum}}^2, \ldots, f_{\text{sum}}^h)$; (iii) when h is fixed, the super-dual FPTAS for $1|\beta^*|\epsilon(f_{\text{sum}}^A, f_{\text{sum}}^B)$ can be extended to solve $1|\beta^*|\epsilon(f_{\text{sum}}^1, f_{\text{sum}}^2, \ldots, f_{\text{sum}}^h)$, while the FPTAS is not applicable; and (iv) when h is arbitrary, new approaches are expected to address these intractable cases. Fourth, in contrast to the scenario where the two agents are competing and compatible, scenarios involving non-disjoint (i.e., $\mathcal{J}^A \cap \mathcal{J}^B \neq \emptyset$) and/or incompatible agents warrant further investigation. Finally, bounded batch scheduling with two agents poses significant theoretical challenges. Unlike the unbounded batch scenario, the ERD property no longer holds, rendering algorithms specifically designed for the unbounded case inapplicable. Instead of pursuing a generic solution, a promising approach would be to tailor efficient exact or heuristic algorithms to specific problems based on the objective functions of the two agents.

Acknowledgements

The authors would like to thank the Editor, Associate Editor, and three anonymous reviewers for their insightful comments and suggestions, which have greatly improved the quality and presentation of this work.

Disclosure statement

No potential conflict of interest was reported by the authors.

Funding

This work was supported by the National Natural Science Foundation of China [grant numbers 12401427, 12471305] and the Key Research Projects of Henan Higher Education Institutions [grant number 24A110014].

Data availability statement

Not relevant for this paper.

References

Agnetis, A., Billaut, J. C., Gawiejnowicz, S., Pacciarelli, D., and Soukhal, A. (2014). *Multiagent Scheduling: Models and Algorithms*. Springer Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-41880-8.

Agnetis, A., Chen, B., Nicosia, G., and Pacifici, A. (2019). Price of fairness in two-agent single-machine scheduling problems. *European Journal of Operational Research*, 276(1):79–87. https://doi.org/10.1016/j.ejor.2018.12.048.

Agnetis, A., Mirchandani, P. B., Pacciarelli, D., and Pacifici, A. (2004). Scheduling problems with two competing agents. *Operations Research*, 52(2):229–242. https://doi.org/10.1287/opre.1030.0092.

Akafuah, N. K., Poozesh, S., Salaimeh, A., Patrick, G., Lawler, K., and Saito, K. (2016). Evolution of the automotive body coating process: A review. *Coatings*, 6(2):24. https://doi.org/10.3390/coatings6020024.

Baker, K. R. and Smith, J. C. (2003). A multiple-criterion model for machine scheduling. *Journal of Scheduling*, 6(1):7–16. https://doi.org/10.1023/A:1022231419049.

Baptiste, P. (2000). Batching identical jobs. *Mathematical Methods of Operations Research*, 52(3):355–367. https://doi.org/10.1007/s001860000088.

Brucker, P., Gladky, A., Hoogeveen, J. A., Kovalyov, M. Y., Potts, C. N., Tautenhahn, T., and van de Velde, S. L. (1998). Scheduling a batching machine. *Journal of Scheduling*, 1(1):31–54. https://doi.org/10.1002/(SICI)1099-1425(199806)1:1<31::AID-JOS4>3.0.CO;2-R.

Cai, L., Li, W., Luo, Y., and He, L. (2023). Real-time scheduling simulation optimisation of job shop in a production-logistics collaborative environment. *International Journal of Production Research*, 61(5):1373–1393. https://doi.org/10.1080/00207543.2021.2023777.

Chen, R. B., Geng, Z. C., Lu, L. F., Yuan, J. J., and Zhang, Y. (2022). Pareto-scheduling of two competing agents with their own equal processing times. *European Journal of Operational Research*, 301(2):414–431. https://doi.org/10.1016/j.ejor.2021.10.064.

Cheng, T. C. E., Liu, Z. H., and Yu, W. C. (2001). Scheduling jobs with release dates and deadlines on a batch processing machine. *IIE Transactions*, 33(8):685–690. https://doi.org/10.1080/07408170108936864.

Cheng, T. C. E., Yuan, J. J., and Yang, A. F. (2005). Scheduling a batch-processing machine subject to precedence constraints, release dates and identical processing times. *Computers & Operations Research*, 32(4):849–859. https://doi.org/10.1016/j.cor.2003.09.001.

Chubanov, S., Kovalyov, M. Y., and Pesch, E. (2006). An FPTAS for a single-item capacitated economic lot-sizing problem with monotone cost structure. *Mathematical Programming*, 106(3):453–466. https://doi.org/10.1007/s10107-005-0641-0.

Dover, O. and Shabtay, D. (2016). Single machine scheduling with two competing agents, arbitrary release dates and unit processing times. *Annals of Operations Research*, 238(1-2):145–178. https://doi.org/10.1007/s10479-015-2054-7.

Fan, B. Q., Cheng, T. C. E., Li, S. S., and Feng, Q. (2013). Bounded parallel-batching scheduling with two competing agents. *Journal of Scheduling*, 16(3):261–271. https://doi.org/10.1007/s10951-012-0274-0.

Fan, G. Q., Wang, J. Q., and Liu, Z. (2023). Two-agent scheduling on mixed batch machines to minimize the total weighted makespan. *International Journal of Production Research*, 61(1):238–257. https://doi.org/10.1080/00207543.2020.18200952.

Fowler, J. W. and Monch, L. (2022). A survey of scheduling with parallel batch (p-batch) processing. *European Journal of Operational Research*, 298(1):1–24. https://doi.org/10.1016/j.ejor.2021.06.012.

Freud, D. and Mosheiov, G. (2021). Scheduling with competing agents, total late work and job rejection. *Computers & Operations Research*, 133:105329. https://doi.org/10.1016/j.cor.2021.105329.

Gao, Y., Yuan, J. J., Ng, C. T., and Cheng, T. C. E. (2019). A further study on two-agent parallel-batch scheduling with release dates and deteriorating jobs to minimize the makespan. *European Journal of Operational Research*, 273(1):74–81. https://doi.org/10.1016/j.ejor.2018.07.040.

Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of $\mathcal{NP}$-Completeness*. W. H. Freeman and Company, San Francisco.

Geng, Z. C. and Yuan, J. J. (2023). Single-machine scheduling of multiple projects with controllable processing times. *European Journal of Operational Research*, 308(3):1074–1090. https://doi.org/10.1016/j.ejor.2023.01.026.

Hall, N. G. and Posner, M. E. (2001). Generating experimental data for computational testing with machine scheduling applications. *Operations Research*, 49(7):854–865. https://doi.org/10.1287/opre.49.6.854.10014.

He, C., Wu, J., and Lin, H. (2022). Two-agent bounded parallel-batching scheduling for minimizing maximum cost and makespan. *Discrete Optimization*, 45:100698. https://doi.org/10.1016/j.disopt.2022.100698.

Huang, J., Wang, L., and Jiang, Z. (2020). A method combining rules with genetic algorithm for minimizing makespan on a batch processing machine with preventive maintenance. *International Journal of Production Research*, 58(13):4086–4102. https://doi.org/10.1080/00207543.2019.1641643.

Ikura, Y. and Gimple, M. (1986). Efficient scheduling algorithms for a single batch processing machine. *Operations Research Letters*, 5(2):61–65. https://doi.org/10.1016/0167-6377(86)90104-5.

Kong, M., Wang, W., Deveci, M., Zhang, Y., Wu, X., and Coffman, D. (2024). A novel carbon reduction engineering method-based deep Q-learning algorithm for energy-efficient scheduling on a single batch-processing machine in semiconductor manufacturing. *International Journal of Production Research*, 62(18):6449–6472. https://doi.org/10.1080/00207543.2023.2252932.

Kovalyov, M. Y., Oulamara, A., and Soukhal, A. (2015). Two-agent scheduling with agent specific batches on an unbounded serial batching machine. *Journal of Scheduling*, 18(4):423–434. https://doi.org/10.1007/s10951-014-0410-0.

Kravchenko, S. A. and Werner, F. (2011). Parallel machine problems with equal processing times: A survey. *Journal of Scheduling*, 14(5):435–444. https://doi.org/10.1007/s10951-011-0231-3.

Kucukkoc, I., Keskin, G. A., Karaoglan, A. D., and Karadag, S. (2024). A hybrid discrete differential evolution-genetic algorithm approach with a new batch formation mechanism for parallel batch scheduling considering batch delivery. *International Journal of Production Research*, 62(1–2):460–482. https://doi.org/10.1080/00207543.2023.2233626.

Lee, C. Y. and Uzsoy, R. (1999). Minimizing makespan on a single batch processing machine with dynamic job arrivals. *International Journal of Production Research*, 37(1):219–236. https://doi.org/10.1080/002075499192020.

Lee, C. Y., Uzsoy, R., and Martin-Vega, L. A. (1992). Efficient algorithms for scheduling semi-conductor burn-in operations. *Operations Research*, 40(4):764–775. https://doi.org/10.1287/opre.40.4.764.

Lei, D. and Dai, T. (2024). An adaptive shuffled frog-leaping algorithm for parallel batch processing machines scheduling with machine eligibility in fabric dyeing process. *International Journal of Production Research*, 62(21):7704–7721. https://doi.org/10.1080/00207543.2024.2324452.

Li, C., Wang, F., Gupta, J. N. D., and Chung, T. (2022). Scheduling identical parallel batch processing machines involving incompatible families with different job sizes and capacity constraints. *Computers & Industrial Engineering*, 169:108115. https://doi.org/10.1016/j.cie.2022.108115.

Li, C. L. and Lee, C. Y. (1997). Scheduling with agreeable release times and due dates on a batch processing machine. *European Journal of Operational Research*, 96(3):564–569. https://doi.org/10.1016/0377-2217(95)00332-0.

Li, S. S. and Chen, R. X. (2023). Competitive two-agent scheduling with release dates and preemption on a single machine. *Journal of Scheduling*, 26(3):227–249. https://doi.org/10.1007/s10951-023-00779-5.

Li, S. S., Chen, R. X., and Tian, J. (2020). Multitasking scheduling problems with two competitive agents. *Engineering Optimization*, 52(11):1940–1956. https://doi.org/10.1080/0305215X.2019.1678609.

Li, S. S. and Yuan, J. J. (2012). Unbounded parallel-batching scheduling with two competitive agents. *Journal of Scheduling*, 15(5):629–640. https://doi.org/10.1007/s10951-011-0253-x.

Liu, L. L., Ng, C. T., and Cheng, T. C. E. (2007). Scheduling jobs with agreeable processing times and due dates on a single batch processing machine. *Theoretical Computer Science*, 374(1-3):159–169. https://doi.org/10.1016/j.tcs.2006.12.039.

Liu, Z. H., Yuan, J. J., and Cheng, T. C. E. (2003). On scheduling an unbounded batch machine. *Operations Research Letters*, 31(1):42–48. https://doi.org/10.1016/S0167-6377(02)00186-4.

Misra, S. and Maravelias, C. T. (2022). *Overview of Scheduling Methods for Pharmaceutical Production*. Springer Berlin, Cham. https://doi.org/10.1007/978-3-030-90924-6_13.

Monch, L., Fowler, J. W., Dauzere-Peres, S., Mason, S. J., and Rose, O. (2011). A survey of problems, solution techniques, and future challenges in scheduling semiconductor manufacturing operations. *Journal of Scheduling*, 14(6):583–599. https://doi.org/10.1007/s10951-010-0222-9.

Nouiri, M., Bekrar, A., and Trentesaux, D. (2020). An energy-efficient scheduling and rescheduling method for production and logistics systems. *International Journal of Production Research*, 58(11):3263–3283. https://doi.org/10.1080/00207543.2019.1660826.

Oron, D. (2021). Two-agent scheduling problems under rejection budget constraints. *Omega*, 102:102313. https://doi.org/10.1016/j.omega.2020.102313.

Oron, D., Shabtay, D., and Steiner, G. (2015). Single machine scheduling with two competing agents and equal job processing times. *European Journal of Operational Research*, 244(1):86–99. http://dx.doi.org/10.1016/j.ejor.2015.01.003.

Perez-Gonzalez, P. and Framinan, J. M. (2014). A common framework and taxonomy for multicriteria scheduling problems with interfering and competing jobs: Multi-agent scheduling problems. *European Journal of Operational Research*, 235(1):1–16. http://dx.doi.org/10.1016/j.ejor.2013.09.017.

Phosavanh, J. and Oron, D. (2024). Two-agent single-machine scheduling with a rate-modifying activity. *European Journal of Operational Research*, 312(3):866–876. https://doi.org/10.1016/j.ejor.2023.08.002.

Rania, M. V. and Mathirajan, M. (2023). A state-of-art review and a simple meta-analysis on deterministic scheduling of diffusion furnaces in semiconductor manufacturing. *International Journal of Production Research*, 61(16):5744–5771. https://doi.org/10.1080/00207543.2022.2102449.

Sabouni, M. T. Y. and Jolai, F. (2010). Optimal methods for batch processing problem with makespan and maximum lateness objectives. *Applied Mathematical Modelling*, 34(2):314–324. https://doi.org/10.1016/j.apm.2009.04.007.

Sahni, S. (1976). Algorithms for scheduling independent tasks. *Journal of the ACM*, 23(1):116–127. https://doi.org/10.1145/321921.321934.

Servranckx, T., Coelho, J., and Vanhoucke, M. (2022). Various extensions in resource-constrained project scheduling with alternative subgraphs. *International Journal of Production Research*, 60(11):3501–3520. https://doi.org/10.1080/00207543.2021.1924411.

Streitberger, H. J. and Dossel, K. F. (2008). *Automotive Paints and Coatings*. Wiley, VCH. https://doi.org/10.1002/9783527622375.

Tang, L., Zhao, X., Liu, J. L., and Leung, J. Y. T. (2017). Competitive two-agent scheduling with deteriorating jobs on a single parallel-batching machine. *European Journal of Operational Research*, 263(2):401–411. https://doi.org/10.1016/j.ejor.2017.05.019.

T'kindt, V. and Billaut, J. C. (2006). *Multicriteria Scheduling: Theory, Models and Algorithms*. Springer Berlin, Heidelberg, second edition. https://doi.org/10.1007/b106275.

Wang, D. J., Yu, Y., Yin, Y., and Cheng, T. C. E. (2021). Multi-agent scheduling problems under multitasking. *International Journal of Production Research*, 59(12):3633–3663. https://doi.org/10.1080/00207543.2020.1748908.

Wang, X., Ren, T., Bai, D., Chu, F., Yu, Y., Meng, F., and Wu, C. C. (2024). Scheduling a multi-agent flow shop with two scenarios and release dates. *International Journal of Production Research*, 62(1–2):421–443. https://doi.org/10.1080/00207543.2023.2188646.

Yu, J., Liu, P., Lu, X., and Gu, M. (2025). Analyzing the price of fairness in scheduling problems with two agents. *European Journal of Operational Research*, 321(3):750–759. https://doi.org/10.1016/j.ejor.2024.10.023.

Zhao, Q. L. and Yuan, J. J. (2020). Bicriteria scheduling of equal length jobs on uniform parallel machines. *Journal of Combinatorial Optimization*, 39(3):637–661. https://doi.org/10.1007/s10878-019-00507-w.

Zhou, S., Jin, M., Liu, C., Zheng, X., and Chen, H. (2022). Scheduling a single batch processing machine with non-identical two-dimensional job sizes. *Expert Systems with Applications*, 201:116907. https://doi.org/10.1016/j.eswa.2022.116907.